
∞ ATOMIC MEMORY ROWS: A BOUNDED, VERIFIABLE SUBSTRATE FOR EDITABLE REASONING

MMLA FOCUS PAPER

Junyi Zou^{1,*} • Avrova Donz^{1,2,*}

¹MMLA-org ²Communication University of China (CUC), No. 1 Dingfuzhuang East Street, Chaoyang District, Beijing 100024, China

*Equal contribution

Email: zoujunyi@zjydiary.cn • avrovadonz@icloud.com

Project: github.com/MMLA-org/mmla-memory

ABSTRACT

Editable reasoning memory needs a unit of authority, not only a place to store vectors. We define an *Atomic Memory Row* (AMR): a fixed-width record in which canonical content, bounded neural payload and routing key, identity, access scope, validity, monotone versions, protection, provenance, tombstone state, bounded rollback lineage, and an authenticated consistency receipt share one serialization and publication boundary. A neural component may propose content and intent; a trusted assembler fixes system-owned fields. Each ordered event replaces one complete row or performs NULL, changing no authoritative bit.

Under explicit assumptions about assembly, deterministic encoding, snapshot-consistent validation, commit serialization, durable publication, and cryptography, we prove fixed resident capacity, invariant preservation, one-row event support, a segment support bound, untouched-row preservation, monotone rollback, logical tenant noninterference, stale-index rejection, serializability, and old-or-new crash recovery. We also prove finite version exhaustion and the impossibility of retaining an unbounded deletion set forever in bounded authoritative state. A reconciled AMR-1 profile occupies 6,592 bytes per row; 256 rows occupy 1,687,552 bytes, with 344,064 additional bytes for two deterministic base indexes. These are protocol counts, not performance measurements. No implementation or semantic-success experiment is reported: earlier same-checkpoint tests qualified 0/9 interfaces, and learned content was exact on 0/73,728 fitted probes even with target routing. Mechanical consistency therefore remains separate from truth, usefulness, learned admission, safety, and hardware advantage.

1 Introduction

1.1 The state problem hidden by the word memory

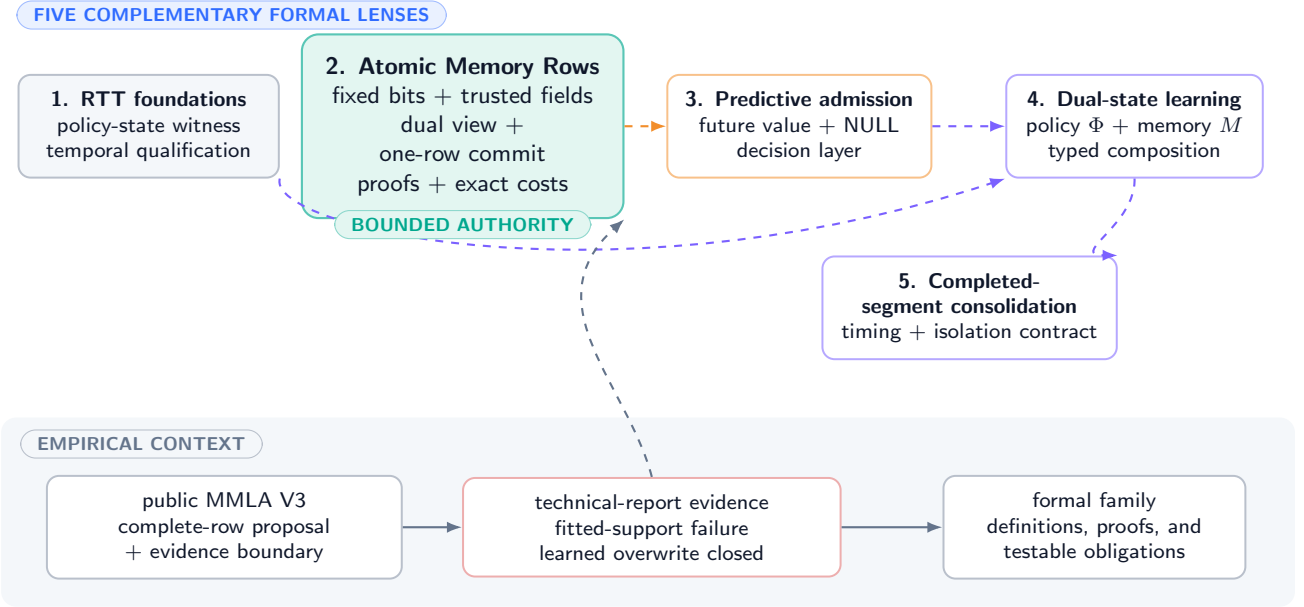
A transcript can retain an observation without deciding whether it is current. A vector store can retrieve neighbors without identifying which record is authoritative. A differentiable memory matrix can change continuously without specifying deletion, provenance, tenant scope, or a version boundary. An append-only audit stream can preserve history while growing without limit. These are useful objects, but none alone is a bounded editable fact state.

The missing object is a complete unit of authority. Suppose a resident record says that Alice is in Paris and a later observation proposes Berlin. Before “write Berlin” is a defined operation, a system must answer at least the following questions. Which logical object and physical slot are involved? Which tenant and principals may read or modify it? Does the proposal replace an entire version or blend with the old value? Are the canonical and neural representations synchronized? Which source and software builds produced the candidate? Does the version advance without wraparound? What happens to stale index entries? Is declining the write distinguishable from deletion? Can rollback restore Paris without restoring obsolete permissions? Which resident, index, archive, audit, and durable-write bytes are charged?

AMR answers those questions at the substrate level. It does not decide whether Berlin is true or useful, and it does not decide whether the proposal deserves admission. The distinction between state correctness and decision quality is the paper’s central evidence boundary.

MMLA / RTT focus-paper family: Atomic Memory Rows in context

conceptual interfaces only — every mechanism retains an independent empirical burden



Dashed arrows denote conceptual dependency only; neither proof nor interface reuse transfers empirical validation.

Figure 1: **Atomic rows occupy the bounded substrate layer of a five-paper conceptual family.** Dependency arrows organize formal interfaces; they do not transfer empirical validation. The semantic-interface and learned-overwrite gates remain closed.

1.2 Authority is narrower than storage

DEFINITION

Definition 1.1 (Authoritative state). A mutable component is *authoritative* if changing it can change the system’s declared current resident object, value, deletion status, security decision, or permitted transition without first validating against a current row. The authoritative resident memory is an ordered table

$$M = (r_1, \dots, r_S) \in \mathcal{R}_\sigma^S, \quad (1)$$

where $S < \infty$ and schema σ fixes the exact row serialization length B_R bytes.

An alias map that can redirect a query without revalidating the current row is authoritative. So is a security map that can grant access independently of the row. Both are outside the contract unless included in the same fixed budget and commit boundary. By contrast, an index entry is a non-authoritative hint when every hit is rechecked against the current row’s tenant, identity, slot sequence, status, and receipt.

We distinguish five state classes.

1. **Transient context** contains tokens, activations, and scratch reasoning for the current computation.
2. **Authoritative resident memory** is the fixed table M and is the only resident state permitted to declare a live, deleted, protected, or quarantined object.
3. **Deterministic derived views** include exact key/alias and neural-routing indexes. They are rebuildable functions of validated rows and never grant authority.
4. **External archives and audit ledgers** may retain source documents, superseded values, or receipts. Their storage and retention policies are separately charged; they are not hidden inside the resident bound.
5. **Model or policy state** includes fixed parameters and any separately typed fast or trainable state. Its mutation is not an AMR row operation.

DESIGN PRESCRIPTION Every implementation must publish a state-carrier inventory assigning each mutable byte to exactly one class, with its capacity, lifetime, authority, and update rule. “Cache,” “metadata,” or “log” is not an exemption from accounting.

1.3 Contributions and scope

This manuscript owns the following formal and systems-contract questions:

- a typed, fixed-width authoritative row and a reconciled concrete profile;
- neural-proposed versus trusted system-owned fields;
- canonical/neural conformance, receipts, re-encoding, and fail-closed mismatch behavior;
- one complete-row commit or bit-identical NULL per ordered event;
- insert, update, consolidating update, eviction, alias rename, deletion, rollback, quarantine, repair, and purge semantics;
- per-event and segment-level edit-locality bounds;
- deterministic derived indexes, stale-entry suppression, tenant partitioning, ACLs, protection, provenance, tombstones, and bounded rollback;
- exact resident, index, candidate, validation, publication, archive, and audit accounting;
- invariant, concurrency, crash-recovery, impossibility, and counterexample results under explicit assumptions; and
- a theorem-to-implementation obligation graph and falsification ladder.

It does not own the event extractor, learned target ranker, predictive admission objective, future-valued controller, archive-retrieval policy, general multi-row transactions, distributed consensus, or end-to-end language-model quality. Those components can consume or produce AMR objects only after their own evidence gates are satisfied.

1.4 Relationship to frozen MMLA evidence

Public MMLA V3 proposed a bounded resident table, a canonical/neural successor row, a trusted assembly boundary, and one overwrite or NULL per event [13]. The subsequent technical-report evidence record sharpened the fixed-bit, event-recursive, dual-view, and complete-cost contract and admitted a stronger negative semantic-interface diagnostic. This paper isolates and extends the substrate mathematics. It does not reinterpret either source as validating AMR.

The empirical ceiling is exact. PM-I2 qualified zero of nine dense, structured-span, and checkpoint-native jobs. Its structural checks passed while semantic prediction failed. A later fitted-population diagnostic reported direct candidate reconstruction 40/73,728, learned route/learned content 0/73,728, oracle route/learned content 0/73,728, learned route/oracle content 18,544/73,728, and the mechanical full oracle 73,728/73,728. These numbers show that target-row isolation did not repair learned content. They do not reject the whole MMLA architecture, and they do not evaluate the fixed-row profile introduced here.

UNVALIDATED No trained checkpoint has been shown here to read an AMR neural capsule semantically, construct a useful complete row, learn a safe admission policy, or improve reasoning. No crash-injection, security, latency, energy, or hardware experiment is reported.

Formal statements carry their assumptions locally. Definition, proof, prescription, and planned-test labels identify the kind of support offered; none is evidence that a checkpoint can use the row. The next sections specify the row schema and exact budget, separate proposed from trusted fields, define dual-view consistency and the transition algebra, then develop lifecycle, invariants, security, recovery, costs, counterexamples, and falsification tests. Whether these bytes improve a reasoner’s behavior remains open.

2 Typed State and Complete-Row Schema

2.1 Finite universes and schema parameters

Fix a schema identifier σ and finite-width spaces

$$\begin{aligned} S &= \{1, \dots, S\}, & \mathcal{T} &= \{0, 1\}^{b_T}, & \mathcal{U} &= \{0, 1\}^{b_U}, \\ O &= \{0, 1\}^{b_O}, & \mathcal{V}_s &= \{0, \dots, 2^{b_s} - 1\}, & \mathcal{V}_o &= \{0, \dots, 2^{b_o} - 1\}. \end{aligned} \quad (2)$$

They denote physical slots, tenants, principals, logical objects, physical-slot sequences, and same-object versions. A trusted static partition

$$P : S \rightarrow \mathcal{T} \quad (3)$$

assigns each slot to exactly one tenant. Dynamic repartitioning is an administrative storage migration with its own transaction; it is not an ordinary memory event.

Schema σ fixes at least the following finite parameters: canonical alphabet and normalization, key and value lengths, alias count and alias length, neural dimensions and precision, ACL capacity, identifier widths, rollback depth, validation bitmap

width, digest and authentication algorithms, time representation, status normal forms, and the exact byte order of every field. A schema version change is a migration, not an in-place reinterpretation.

DEFINITION

Definition 2.1 (Status). Every parsed row has one status

$$z(r) \in \{\text{EMPTY}, \text{LIVE}, \text{TOMBSTONE}, \text{QUARANTINED}\}. \quad (4)$$

An EMPTY row has no logical object but retains physical slot, tenant, slot sequence, and a valid empty-row receipt. A LIVE row may supply content to an authorized reader. A TOMBSTONE row records a committed deletion and is unavailable to ordinary semantic reads. A QUARANTINED row records a bounded integrity failure and is unavailable until an authorized repair commits a new complete version.

EMPTY is not deletion, TOMBSTONE is not NULL, and QUARANTINED does not assert that the logical object was deleted.

2.2 The row product type

DEFINITION

Definition 2.2 (Atomic Memory Row). An Atomic Memory Row is the product

$$r = (h, c, n, p, \lambda, \rho) \in \mathcal{H}_\sigma \times \mathcal{C}_\sigma \times \mathcal{N}_\sigma \times \mathcal{P}_\sigma \times \mathcal{L}_\sigma \times \mathcal{Q}_\sigma, \quad (5)$$

where h is the trusted control block, c the canonical capsule, n the neural capsule, p the provenance capsule, λ the bounded rollback lineage, and ρ the consistency receipt. Schema σ defines a deterministic serializer and partial parser

$$\text{Ser}_\sigma(r) \in \{0, 1\}^{8BR}, \quad \text{Parse}_\sigma : \{0, 1\}^{8BR} \rightarrow \mathcal{R}_\sigma \cup \{\perp\}. \quad (6)$$

The row is the publication unit: no field in Eq. 5 is authoritative on a different commit boundary.

The serializer is injective over valid rows. Lengths, endianness, normalization, padding, ordering, floating or quantized numeric semantics, reserved-bit values, and prohibited encodings are part of σ . Deterministic encoding recommendations such as CBOR’s deterministic profiles are useful background [1], but AMR requires one schema-specific bit string rather than merely equivalent decoded maps.

2.3 Trusted control block

The control block is

$$h = (\text{magic}, \sigma, z, \text{committed-op}, \text{flags}, a, t, o, u, \tau, v_s, v_o, t_{\min}, t_{\max}, t_{\text{retain}}, \pi, \delta, \text{ACL}, q_{\text{rb}}), \quad (7)$$

where a is physical slot, t tenant, o object, u owner, τ object type, v_s slot sequence, v_o object version, the three times encode validity and retention, π is protection class, δ a deletion-reason code, and q_{rb} describes rollback-ring occupancy. The ACL is a fixed array of principal/right pairs. Its empty entries have a unique zero representation.

The slot sequence advances on every committed replacement of a physical slot. The object version advances when the same logical object remains resident. Insertion of a different object starts its object version at one while still advancing the slot sequence. This two-counter design makes stale physical pointers distinguishable from a new object that reuses the same slot.

2.4 Canonical capsule

The canonical capsule is

$$c = (\tau_c, k_c, v_c, A_c, f_c), \quad (8)$$

where τ_c is a type tag, k_c a typed key, v_c a typed value or canonical tuple, A_c an ordered bounded alias array, and f_c semantic flags. The reference profile places the authoritative type and flags in the trusted control block while the canonical byte region contains the normalized key, value, and aliases. Their joint interpretation is nevertheless one canonical capsule.

Canonical means inspectable and deterministically serialized, not factually true. Unicode normalization, case handling, locale, units, temporal interpretation, numeric canonicalization, and missing-value semantics are fixed by schema. A canonical $\emptyset$ is data with a nonzero type/length encoding. Padding bytes are not content.

Aliases are alternate query handles for the same row identity. They do not create extra logical objects or independently mutable mappings. Alias rename is a complete same-object row update, and every derived alias-index hit is revalidated against the current row.

2.5 Neural capsule

The neural capsule is

$$n = (k_n, v_n, u_n, \alpha_n, \tilde{v}), \quad (9)$$

where k_n is a fixed-dimensional routing vector, v_n a fixed-dimensional semantic payload, u_n bounded uncertainty metadata, α_n a bounded abstention score, and $\tilde{v}$ proposed operation intent. The schema chooses a finite coordinate representation. AMR-1 uses signed 16-bit coordinates. A floating-point profile must fix byte order, rounding, negative zero, infinities, denormals, and NaNs; unrestricted NaN payloads are prohibited because bitwise equality and receipt hashing would otherwise be noncanonical.

The neural capsule is not authoritative by itself. A matching vector cannot override canonical content, tenant scope, a tombstone, or a failed receipt. A separately typed differentiable fast state may use soft reads and writes, but Section 6 prevents it from silently becoming the versioned fact table.

2.6 Provenance, rollback, and receipt

The provenance capsule records bounded identifiers or digests for the event, source material, proposer/model build, assembler, encoder, validator, observation time, source class, committed operation, and deletion authorization. Provenance concepts distinguish where a datum came from and how it was transformed [2]; the AMR capsule intentionally stores only a finite receipt, not an unbounded derivation graph.

PROVED UNDER STATED ASSUMPTIONS Under the cryptographic assumptions in Section 8.1, provenance can bind declared source and build identifiers to committed bytes. It cannot establish that the source was truthful, that the validator was competent, or that the neural representation is useful.

Rollback lineage λ is a ring of at most H_{rb} fixed-width same-object snapshots. A snapshot contains the canonical and neural semantic state plus bounded prior version, validity, protection, and digest metadata, but contains no nested rollback lineage. A different object evicted from a slot is not placed in the new object’s rollback ring. Recovering it requires a separately charged archive or a new source event.

Let $\text{core}(r)$ be every serialized row byte except the receipt. For a hash H and authentication scheme Tag_K , define

$$d_{\text{core}} = H(\text{AMR-CORE} \parallel \text{core}(r)), \quad (10)$$

and

$$\rho = (d_{\text{core}}, d_{\text{prev}}, b_{\text{val}}, \text{alg-ids}, \text{event-bind}, \text{Tag}_K(m), \text{reserved}), \quad (11)$$

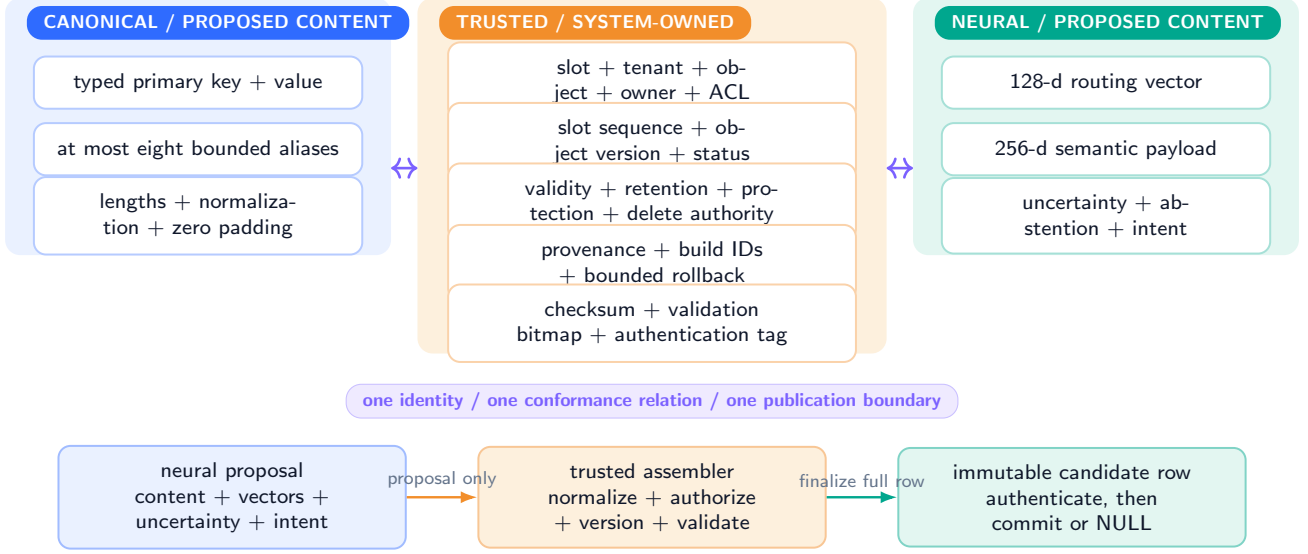
where

$$m = H(\text{AMR-RECEIPT} \parallel d_{\text{core}} \parallel d_{\text{prev}} \parallel \text{event-bind} \parallel b_{\text{val}} \parallel \text{assembler-id}). \quad (12)$$

The digest is the row’s cryptographic checksum; the authentication tag may be a MAC such as HMAC or a signature according to the trust model [11, 10]. The receipt authenticates bytes and declared checks under stated assumptions. It is not a zero-knowledge proof, an attestation that the source is true, or evidence that the validator implements the intended semantics.

One complete row: proposal boundary, trusted spine, and one receipt

DEFINITION / AMR-1 PROFILE UNVALIDATED Every field is fixed-width and publishes under one version



The model never owns tenant, ACL, final versions, protection weakening, deletion authority, rollback lineage, checksum, or receipt.

Figure 2: **Complete-row anatomy and ownership.** The figure defines one authoritative serialization. Neural components may propose bounded content, vectors, uncertainty, and intent; trusted code fixes identity, security, versions, policy, lineage, deletion authorization, validation state, and the authenticated receipt. Both views and every control field publish together or none becomes authoritative.

2.7 Normal forms and missing values

Status normal forms eliminate ambiguous interpretations.

- **EMPTY** has zero logical object, zero active canonical/neural regions, tenant and slot binding, current slot sequence, and a valid receipt.
- **LIVE** has a nonzero object, a well-formed canonical capsule, a conforming neural capsule, and readable status subject to current policy.
- **TOMBSTONE** retains object identity, versions, deletion authorization, retention, and receipt while active value and read payload regions take the schema’s tombstone normal form.
- **QUARANTINED** retains only the trusted recoverable identity and bounded failure metadata permitted by policy; its active content is unavailable.

A schema-level optional value is encoded with an explicit presence bit or type tag. It is never inferred from an all-zero payload. **NULL** is an action, not a nullable value. This distinction becomes formal in Section 6.

3 Exact Bit and Resource Budgets

3.1 A symbolic finite contract

Let a schema allocate byte widths

$$B_R = B_h + B_c + B_n + B_p + B_A + B_\rho. \quad (13)$$

Exactly S rows therefore consume

$$B_{\text{auth}} = SB_R \text{ bytes}, \quad \text{bits}(M) = 8SB_R. \quad (14)$$

This is operational only if every summand is finite and includes the fields required to interpret and authorize a row. “ S objects” is not a bound when an object can carry an unbounded alias list, provenance graph, source text, or rollback history.

Let B_I be fixed materialized-index capacity, B_Q a bounded diagnostic quarantine buffer, B_F separately typed non-authoritative fast state, $B_A(t)$ archive bytes retained at time t , and $B_L(t)$ audit-ledger bytes. Then

$$B_{\text{hot}} = SB_R + B_I + B_Q + B_F, \quad (15)$$

while

$$B_{\text{system}}(t) = B_{\text{hot}} + B_A(t) + B_L(t) + B_{\text{replica}}(t) + B_{\text{fs}}(t). \quad (16)$$

The resident contract bounds Eq. 15. A deployment may also cap every term in Eq. 16; otherwise it must say that total retained storage is unbounded. A finite row digest that points to an ever-growing archive does not make the archive finite.

3.2 AMR-1: a reconciled concrete profile

DESIGN PRESCRIPTION AMR-1 is an illustrative auditable profile, not an optimized standard and not an implemented format.

Block	Exact allocation	Bytes
Trusted control	identity, status, operation, versions, times, protection, fixed ACL, rollback pointers	256
Canonical capsule	key, value, eight aliases, lengths, deterministic padding	1,104
Neural capsule	128-dimensional route, 256-dimensional payload, uncertainty, abstention, intent	784
Provenance	event/source/model/build/time/deletion-authorization identifiers and digests	192
Rollback	two nonrecursive, same-object snapshots of 2,048 bytes each	4,096
Receipt	core/predecessor digests, validation bitmap, algorithms, event binding, auth tag	160
Complete row	one fixed serialization and publication unit	6,592

Table 1: AMR-1 exact top-level row allocation. Appendix B reconciles every subfield.

Thus

$$B_R = 6,592 \text{ bytes} = 52,736 \text{ bits}. \quad (17)$$

For $S = 256$,

$$B_{\text{auth}} = 256 \times 6,592 = 1,687,552 \text{ bytes} = 1.609375 \text{ MiB}. \quad (18)$$

The count excludes proposal buffers, model state, archives, filesystem metadata, allocation overhead, replicas, and diagnostic logs. Exclusion is not disappearance: Section 10 charges each separately.

3.3 Fixed deterministic base indexes

AMR-1 defines two optional, materialized, slot-major indexes. The canonical index stores one 112-byte entry for the primary key and each of eight aliases in every slot:

$$|I_C| = 256 \times 9 \times 112 = 258,048 \text{ bytes}. \quad (19)$$

The neural routing index stores one 336-byte entry per slot:

$$|I_N| = 256 \times 336 = 86,016 \text{ bytes}. \quad (20)$$

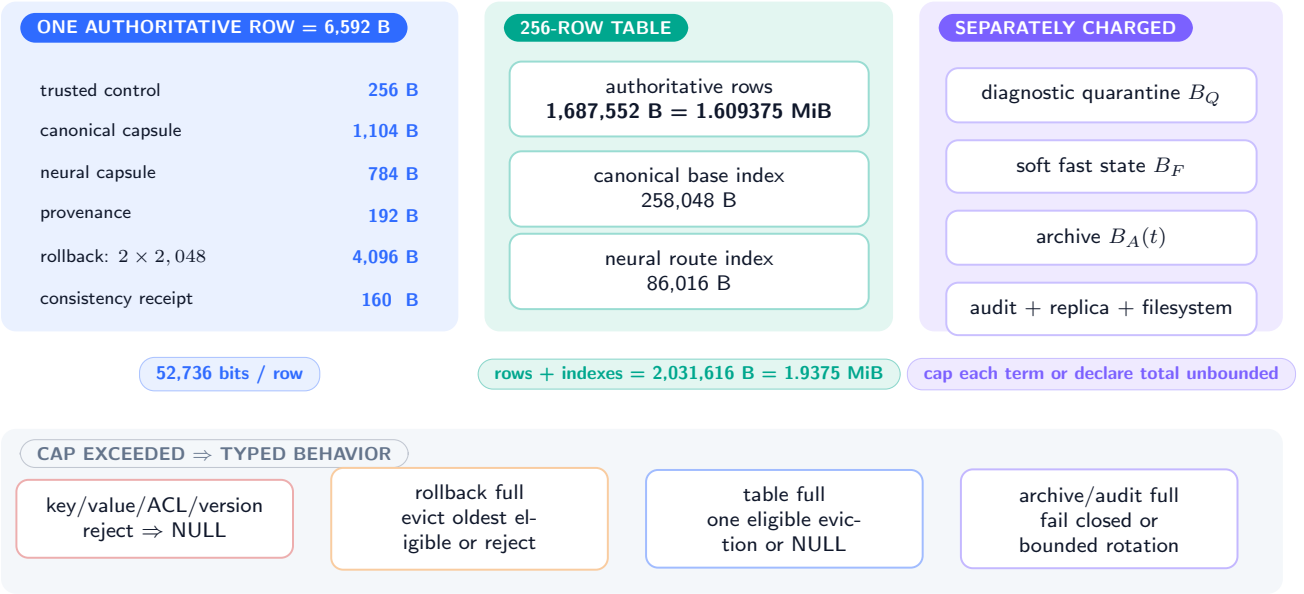
Hence

$$B_I = 344,064 \text{ bytes}, \quad B_{\text{auth}} + B_I = 2,031,616 \text{ bytes} = 1.9375 \text{ MiB}. \quad (21)$$

Each index entry carries tenant, object, slot, slot sequence, and receipt digest. A canonical entry additionally carries a key digest and alias ordinal; a neural entry carries the quantized routing vector. Index capacity cannot overflow because every slot owns a fixed projection. Optional trees, graphs, approximate-nearest-neighbor structures, and learned routers require separate caps and miss accounting.

AMR-1 finite bit ledger: resident authority, derived views, and external history

EXACT PROTOCOL ALLOCATION / NOT PERFORMANCE. All values are logical bytes



No semantic truncation, hidden slot, independently mutable alias authority, or uncharged append-only history.

Figure 3: **Finite resident accounting versus separately charged history.** AMR-1 fixes every authoritative row and base-index byte. Quarantine and fast state require explicit caps. Archive, audit, replication, and filesystem terms are either independently bounded or declared unbounded; they never disappear behind the resident-table notation.

3.4 Overflow and cap semantics

DESIGN PRESCRIPTION Every cap has a typed action. Silent allocation, silent semantic truncation, and an implicit spill that changes authority are prohibited.

Exceeded resource	AMR-1 action	Reason
Canonical key/value length	reject candidate; event returns NULL	Truncation can change object identity or meaning.
Alias count or alias length	strict profile rejects; a different schema may drop only explicitly nonessential aliases and must receipt the loss	Alias loss must never be silent or alter the primary key.
Neural dimension/precision	reject candidate	Reallocation would change schema and digest interpretation.
ACL capacity	reject ACL mutation or use an explicitly bounded external capability system whose decision is re-bound into the row	Silent principal loss can grant or deny access.
Provenance document	store only fixed digest/identifier; optional source document spills to separately charged archive	The row never embeds an unbounded provenance graph.
Rollback ring	deterministically evict the oldest eligible snapshot; if policy forbids loss, reject the new commit	History loss is explicit and bounded.
Resident table capacity	choose one eligible eviction target and replace it completely; otherwise NULL	A hidden extra slot would violate the table bound.
Version counter	return NULL with F_VERSION_EXHAUSTED; external re-key/migration required	Counters never wrap, saturate, or reuse an identity/version pair.
Base index	impossible under the fixed slot-major layout; corrupt indexes are discarded and rebuilt	Derived views cannot force an authoritative mutation.
Optional index	evict/rebuild/disable within its own cap; never accept an unvalidated hit	Index failure may hurt recall or latency, not authority.
Quarantine diagnostics	drop non-authoritative diagnostic detail under a bounded policy; resident reads still fail closed	Diagnostic capacity cannot make corrupt content readable.
Required archive receipt	if the archive cap is full, reject commits whose policy requires archival retention	A required durability/provenance premise cannot be silently weakened.
Required audit ledger	fail closed before publication, or rotate through an authenticated bounded check-point protocol	An unaudited commit is not equivalent to an audited one.

Table 2: Cap behavior. NULL is the authoritative result of precommit rejection; diagnostic reasons live in a separately bounded channel.

3.5 Finite capacity theorem

PROVED UNDER STATED ASSUMPTIONS

Theorem 3.1 (Fixed information capacity). *For a schema with S rows of exactly $8B_R$ bits,*

$$|\mathcal{M}| \leq 2^{8SB_R}. \quad (22)$$

For every random history $X_{\leq t}$ and resulting authoritative memory M_t ,

$$H(M_t) \leq 8SB_R, \quad I(M_t; X_{\leq t}) \leq 8SB_R. \quad (23)$$

Proof. The complete table is a bit string of exactly $8SB_R$ bits, although only a subset may parse as valid. It therefore has at most 2^{8SB_R} possible values and entropy at most $8SB_R$. Mutual information is bounded by the entropy of either argument. The theorem concerns authoritative resident state only; adding external archives increases the corresponding system capacity. $\square$

The theorem does not say that all bits are efficiently used, that the retained information is predictive, or that a language model can read it.

4 Neural Proposal and Trusted Assembly

4.1 The proposal is not a row

Let a neural constructor receive a completed event e , bounded context summary z , target slot a , prior row r_a , and a read-only memory snapshot M . Its output is

$$\tilde{p} = G_\phi(z, e, a, r_a, M) = (\tilde{c}, \tilde{n}, \tilde{t}, \tilde{q}, \tilde{s}), \quad (24)$$

where $\tilde{c}$ is proposed canonical content, $\tilde{n}$ a proposed neural capsule, $\tilde{t}$ operation intent, $\tilde{q}$ uncertainty/abstention, and $\tilde{s}$ bounded source handles. This object is untrusted and may be malformed, adversarial, stale, or semantically wrong.

The trusted assembler is a partial function

$$\text{Assemble}_\sigma(M, a, \tilde{p}, \chi) \in \mathcal{R}_\sigma \cup \{\perp\}, \quad (25)$$

where authenticated context χ binds tenant, principal, capabilities, event identifier, source-authentication result, and snapshot epoch. Returning $\perp$ means that no authoritative candidate exists; the event resolves to NULL.

4.2 Field ownership

DESIGN PRESCRIPTION

Field family	Neural role	Trusted/system role
Canonical key, value, aliases, type	propose bounded content	normalize, type-check, resolve identity, enforce caps and uniqueness
Routing vector and semantic payload	propose or request deterministic encoding	encode/re-encode or validate exact conformance; freeze encoder identity
Uncertainty, abstention, requested operation	propose signals and intent	interpret under fixed policy; choose committed operation/status
Target slot	score or propose candidate target	verify partition, occupancy, eviction eligibility, and snapshot freshness
Tenant, owner, ACL	no authority	authenticate and assemble; require explicit capability for ACL change
Object identity	suggest reference only	allocate or resolve stable identifier and same-object relation
Slot sequence and object version	no authority	increment exactly; reject stale snapshot or overflow
Validity and retention	suggest evidence times only	map to policy-approved intervals and legal holds
Protection	request change only	inherit/strengthen; require capability to weaken
Deletion/tombstone	request intent only	verify delete/resurrection authority and deletion receipt
Provenance	provide bounded source handles	authenticate sources; bind event and build identifiers/digests
Rollback lineage	no authority	push eligible same-object snapshot; apply bounded eviction policy
Checksum, validation bitmap, auth tag	no authority	compute after every check over the final serialized core

Table 3: Field ownership. A neural proposal cannot grant itself security scope, reset lineage, weaken protection, authorize deletion, or authenticate its own bytes.

System ownership is functional, not merely documentary. If a model can choose an arbitrary bit string that the assembler copies into the tenant or version field, then the model owns that field regardless of the API name.

4.3 Assembler sequence

For one target-conditioned proposal, the reference assembler executes:

1. authenticate χ and bind the request to one snapshot epoch;
2. verify $P(a) = \chi.tenant$ and target feasibility;
3. parse the requested operation and resolve whether the old and proposed records represent the same logical object;
4. normalize canonical bytes and enforce every key, value, alias, type, and padding bound;
5. derive or validate the neural capsule under the declared encoder/validator profile;
6. evaluate ACL, protection, validity, retention, deletion, resurrection, eviction, and legal-hold policies;
7. compute the next slot sequence and object version, rejecting stale inputs and overflow;
8. assemble fixed provenance and update the bounded same-object rollback ring;
9. serialize the complete core in canonical order;
10. evaluate row-local and memory-global invariants against the same snapshot;
11. compute the checksum, predecessor binding, validation bitmap, and authentication tag; and
12. return one immutable candidate together with a compare-and-swap precondition.

Any failure returns a typed diagnostic outside authoritative state and no candidate. Candidate validation is not a menu from which the controller may select a partially checked row.

4.4 Local and global validation

Row-local predicates check parsing, status normal form, field bounds, tenant/slot binding, versions, ACL encoding, protection, retention, dual-view conformance, rollback bounds, provenance completeness, reserved bytes, checksum, and authentication. Memory-global predicates check at least uniqueness of live/tombstoned object identifiers and primary typed keys within a tenant. A strict schema may also reject alias collisions; a permissive schema returns a set of candidates and still requires current-row revalidation.

Global validation must be snapshot consistent. Suppose two concurrent inserts both observe that key k is absent and then commit to different slots. Per-row compare-and-swap alone permits a duplicate. The contract therefore requires either one

total commit order per tenant, a tenant-epoch compare-and-swap, or a serializable uniqueness index whose update is part of the same transaction. Section 9 makes this assumption explicit.

4.5 No self-authentication

PROVED UNDER STATED ASSUMPTIONS

Proposition 4.1 (Trusted-field nondelegation). *Assume the assembler computes every trusted field as a function of authenticated context, the current validated snapshot, fixed policy, and validated proposal content, and assume the proposer cannot forge authentication. Then changing only the neural proposal cannot directly choose tenant, ACL, final versions, protection weakening, deletion authority, rollback lineage, or a valid receipt outside the image permitted by those trusted functions.*

Proof. By construction, none of the named output coordinates is copied from an unconstrained proposal coordinate. Each is either fixed by authenticated context and snapshot, derived by deterministic policy, or rejected. A proposal can alter a trusted field only through an explicitly authorized request whose policy preconditions hold. A receipt authenticating an otherwise forbidden row would require either assembler deviation or authentication forgery, both excluded by assumption. $\square$

This proposition is a software-boundary theorem. It does not prove that a particular assembler is bug-free, that its authentication is uncompromised, or that policy is ethically or legally correct.

5 Canonical–Neural Consistency

5.1 Consistency is a relation, not co-location

Let c be canonical content, (k_n, v_n) the routing and semantic neural bytes, ω a frozen encoder artifact, and Q_σ a deterministic quantizer/serializer. The base AMR-1 conformance relation is

$$D_{\sigma, \omega}(c, n) = \text{PASS} \iff \text{Ser}_{k, v}(n) = \text{Ser}_{k, v}(Q_\sigma(E_\omega(\text{Can}_\sigma(c)))) . \quad (26)$$

The row receipt binds one identity and version to the canonical digest, declared encoder digest, derived neural digest, resident neural digest, validation bitmap, and authentication tag. Digest equality accelerates audit, but the contract’s equality predicate is over the fixed serialized neural bytes. A hash collision cannot convert unequal bytes into a valid base-profile row because the validator recomputes Eq. 26.

DEFINITION

Definition 5.1 (Dual-view validity). A LIVE candidate is dual-view valid when (i) its canonical bytes parse and normalize uniquely; (ii) its neural route and payload have the exact declared shape and finite representation; (iii) the registered conformance relation returns PASS; (iv) identity, version, encoder/validator IDs, and security scope agree across the receipt and control block; and (v) every required semantic qualification bit in the validation bitmap is present. A missing check is not a pass.

The last condition is an interface gate, not a theorem. A profile may distinguish mechanical byte equality from empirical semantic qualification. AMR-1 can mechanically derive a vector even when no checkpoint can use it. Such a profile remains **UNVALIDATED** for semantic readout until reconstruction, query, calibration, preservation, and stale-suppression tests pass.

5.2 Certified-proposal profiles

A non-derived profile may permit the proposer to supply n and use a fixed validator

$$D_{\sigma, \omega}(c, n) \in \{\text{PASS}, \text{FAIL}, \text{UNKNOWN}\} . \quad (27)$$

The validator may combine deterministic decoding, typed relation checks, bounded distances, or a frozen semantic reader. Only PASS may produce a LIVE row. FAIL and UNKNOWN fail closed. The receipt records the exact validator and encoder builds; upgrading either creates a new schema or a versioned re-encoding event.

DESIGN PRESCRIPTION A learned validator must not be described as a proof. It is a component whose false-accept and false-reject behavior requires fresh, adversarial, and cross-implementation evidence. A receipt can make the responsible build attributable without making it correct.

5.3 Write, read, load, and migration obligations

The same relation is checked at more than one point.

- **Write.** The complete candidate is validated after final canonicalization and neural serialization, before receipt creation and action selection.

- **Commit.** Canonical bytes, neural bytes, encoder/validator IDs, validation bitmap, and receipt publish atomically.
- **Read.** The current row’s status, checksum, authentication, tenant/ACL, validity, and conformance are checked, either directly or through a cached verification whose key includes the exact slot sequence and receipt digest.
- **Load/recovery/export.** Rows are parsed and revalidated before they enter a readable table or serve as teacher state.
- **Index rebuild.** Derived canonical and neural entries are emitted only from validated current rows.
- **Refresh/re-encoding.** A changed encoder, quantizer, or payload is a new complete-row event with new versions and receipt; no in-place neural drift is legal.

For request context χ , define

$$\begin{aligned} \text{Read}(r, \chi) = c(r) \text{ or } n(r) \quad \text{only if} \quad & z(r) = \text{LIVE}, \text{ReceiptOK}(r) = 1, \\ & D_{\sigma, \omega}(c(r), n(r)) = \text{PASS}, \\ & \text{AuthorizedRead}(r, \chi) = 1, \\ & \text{TimeValid}(r, \chi) = 1. \end{aligned} \quad (28)$$

Otherwise $\text{Read}(r, \chi) = \perp$. No fallback chooses whichever view happens to decode.

Canonical–neural consistency across write, commit, read, and migration

MECHANICAL CONSISTENCY FORMALIZED / SEMANTIC USE UNVALIDATED

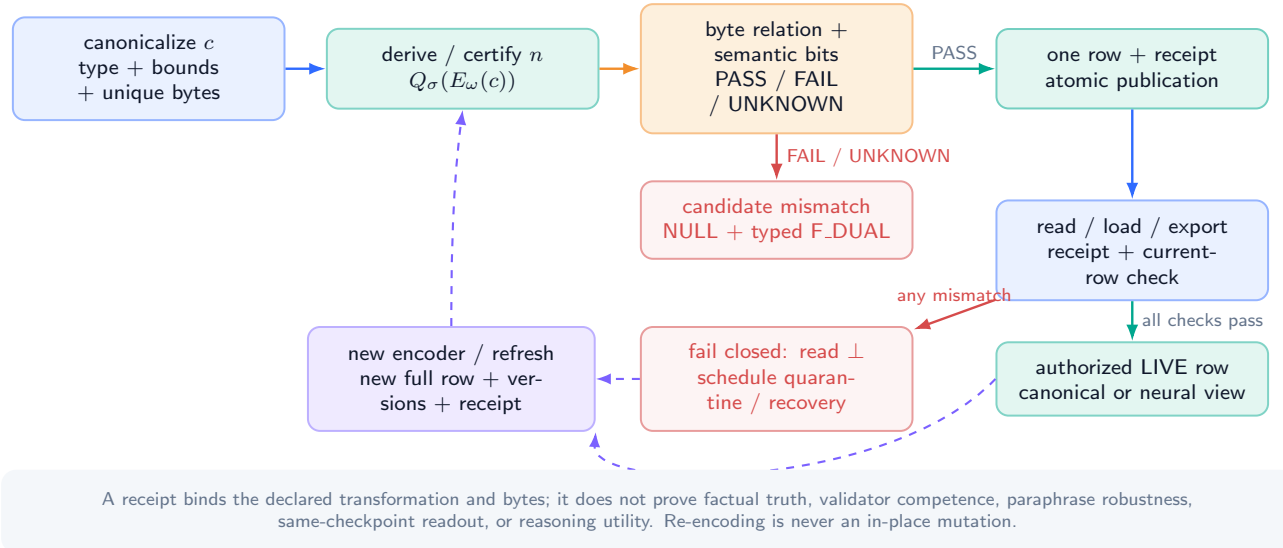


Figure 4: **Dual-view conformance is checked across the row lifecycle.** A candidate mismatch produces NULL; a resident mismatch suppresses the read and schedules a separately authorized quarantine or repair. Re-encoding is a new versioned commit. Mechanical equality is formalized; semantic usefulness remains an unpassed empirical gate.

5.4 Mismatch and quarantine

Candidate-time mismatch has the authoritative result

$$M' = M, \quad \text{reason} = \text{F_DUAL}, \quad (29)$$

with an optional copy in a separately bounded non-authoritative diagnostic buffer. That copy may never be read as fact memory.

A mismatch discovered in a previously committed row can arise from storage corruption, lost encoder artifacts, a bug, or compromised infrastructure. The immediate read result is $\perp$; the read path itself does not silently mutate authoritative state. A maintenance event may replace the same slot with a valid QUARANTINED row if identity and version lineage can be recovered under trusted storage metadata. If the row receipt or version is corrupt beyond safe recovery, repair requires an authenticated replica, copy-on-write root, or external recovery record. A checksum detects inconsistency; it does not invent the lost state.

PROVED UNDER STATED ASSUMPTIONS

Theorem 5.2 (No torn dual-view authority). *Assume one atomic publication exposes either the complete old row or the complete new row. A legal commit cannot expose the new canonical view with the old neural view, or the old canonical view with the new neural view.*

Proof. Both views are byte regions of one fixed row serialization. Atomic publication has exactly two visible row values: the precommit serialization and the postcommit serialization. Neither mixed serialization is in that set. $\square$

Without atomic publication, a receipt can detect a tear after the fact but does not guarantee automatic recovery. Section 9 states the stronger storage assumption needed for crash-safe old-or-new recovery.

5.5 What conformance does and does not prove

PROVED UNDER STATED ASSUMPTIONS In the derived profile, equality proves that the resident neural bytes are the declared deterministic encoder/quantizer output on the resident canonical bytes.

PROVED UNDER STATED ASSUMPTIONS In a certified profile, an authentic receipt proves only that the declared validator returned PASS on the committed bytes, assuming the validator output and receipt path were faithfully implemented.

UNVALIDATED Neither statement proves factual truth, semantic sufficiency, robustness to paraphrase, downstream reasoning improvement, same-checkpoint readout, or learned admission quality. The negative frozen MMLA evidence is precisely why this boundary must remain visible.

6 Event-Recursive Transition Algebra

6.1 One row or none

For memory M , target a , and complete candidate $\hat{r}_a$, define

$$\text{Replace}(M, a, \hat{r}_a)_i = \begin{cases} \hat{r}_a, & i = a, \\ r_i, & i \neq a. \end{cases} \quad (30)$$

DEFINITION

Definition 6.1 (NULL). The action NULL is exact state identity:

$$T_{\text{NULL}}(M) = M \quad (31)$$

bit for bit. It increments no version, creates no row receipt, changes no index generation, writes no tombstone, and clears no payload. A separately charged decision log may record a rejected attempt, but that log is not authoritative resident memory.

For one validated event, the authoritative transition is

$$T(M, e, \chi, a, \hat{r}_a) = \begin{cases} M, & a = \emptyset \text{ or validation fails,} \\ \text{Replace}(M, a, \hat{r}_a), & \text{authorized commit.} \end{cases} \quad (32)$$

An implementation can use copy-on-write pages, a write-ahead log, or an indirection root. Those are physical mechanisms for exposing the logical old-or-new row; they do not create a second authoritative update operator.

6.2 Completed segments and ordered events

Let completed segment n be eventized into

$$E_n = (e_{n,1}, \dots, e_{n,K_n}), \quad 0 \leq K_n \leq K_{\max}. \quad (33)$$

The within-segment state recursion is

$$M_{n,0} = M_{n-1}, \quad M_{n,j} = T_{n,j}(M_{n,j-1}), \quad M_n = M_{n,K_n}. \quad (34)$$

Every event is assembled and validated against its actual branch prefix $M_{n,j-1}$. A candidate computed for $M_{n,0}$ cannot be reused at event j after earlier commits unless its snapshot preconditions are revalidated.

Let

$$C_n = \sum_{j=1}^{K_n} \mathbf{1}[M_{n,j} \neq M_{n,j-1}] \quad (35)$$

be the number of accepted commits and let D_n be the number of distinct physical slots changed at least once. Repeated updates to one slot count once in D_n and multiple times in C_n .

PROVED UNDER STATED ASSUMPTIONS

Theorem 6.2 (Event and segment edit locality). *For every legal event j ,*

$$|\text{supp}_{\text{row}}(M_{n,j} - M_{n,j-1})| \leq 1, \quad d_H(M_{n,j}, M_{n,j-1}) \leq 8B_R. \quad (36)$$

Across the segment,

$$C_n \leq K_n, \text{quad} D_n \leq \min(C_n, S), \quad |\text{supp}_{\text{row}}(M_n - M_{n-1})| \leq D_n \leq K_n. \quad (37)$$

If $V(M) \in \mathbb{R}^{S \times d_v}$ stacks decoded payloads, then

$$\text{rank}(V(M_{n,j}) - V(M_{n,j-1})) \leq 1 \quad (38)$$

for one event, while only the correct segment bound

$$\text{rank}(V(M_n) - V(M_{n-1})) \leq \min(D_n, d_v) \quad (39)$$

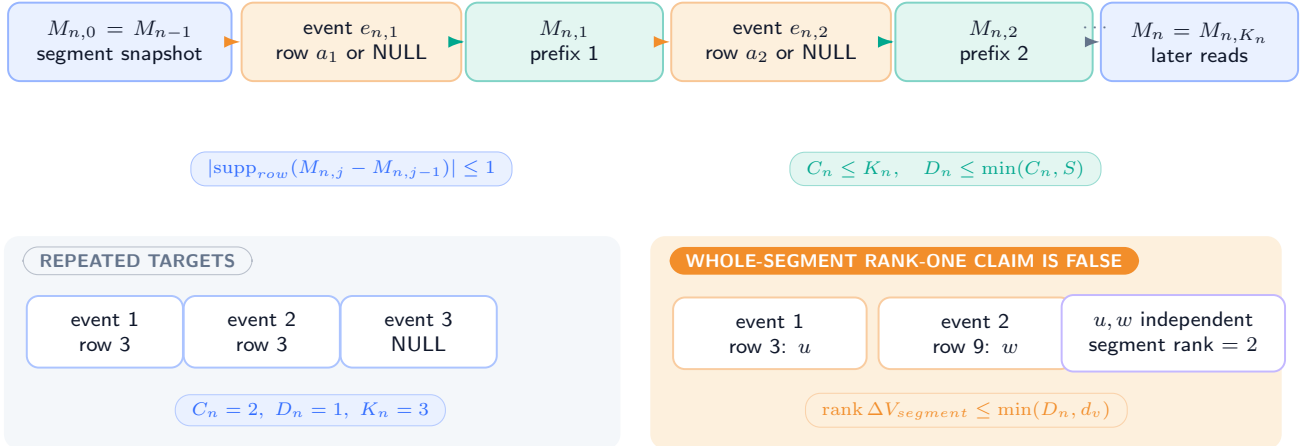
is generally valid.

Proof. NULL changes no row. A committed event replaces exactly target a , so every other row difference is zero and the Hamming distance is at most the complete row width. Each event contributes at most one commit and one target to the union of touched slots; therefore $C_n \leq K_n$ and $D_n \leq \min(C_n, S)$. The final difference is supported only on that union. A matrix with at most one nonzero row has rank at most one; across the segment it may have D_n nonzero rows, so rank is bounded by both D_n and d_v . $\square$

The theorem does not say that a row edit is semantically small. Nor does it say that a whole segment has rank one. With two events updating linearly independent payload rows, the segment difference can have rank two.

Ordered event recursion: atomic per event, bounded support per segment

PROVED UNDER STATED ASSUMPTIONS later events validate against the actual committed prefix



A segment is prefix-committing: an earlier accepted event remains even if a later event returns NULL.

Figure 5: **Event-level atomicity composes into a segment support bound, not segment rank one.** Each event reads its current prefix, then commits one row or NULL. The union of changed targets has size at most K_n ; repeated targets reduce the distinct-row count.

6.3 Hard authority and separately typed fast state

An authoritative update cannot be a soft interpolation

$$r'_a = (1 - \gamma)r_a + \gamma\hat{r}_a, \text{quad} 0 < \gamma < 1, \quad (40)$$

because object IDs, ACLs, tombstone status, versions, type tags, and serialized canonical strings do not have a policy-independent convex meaning. Even restricting interpolation to the neural payload creates a third state unless its relation to the canonical version is revalidated and committed.

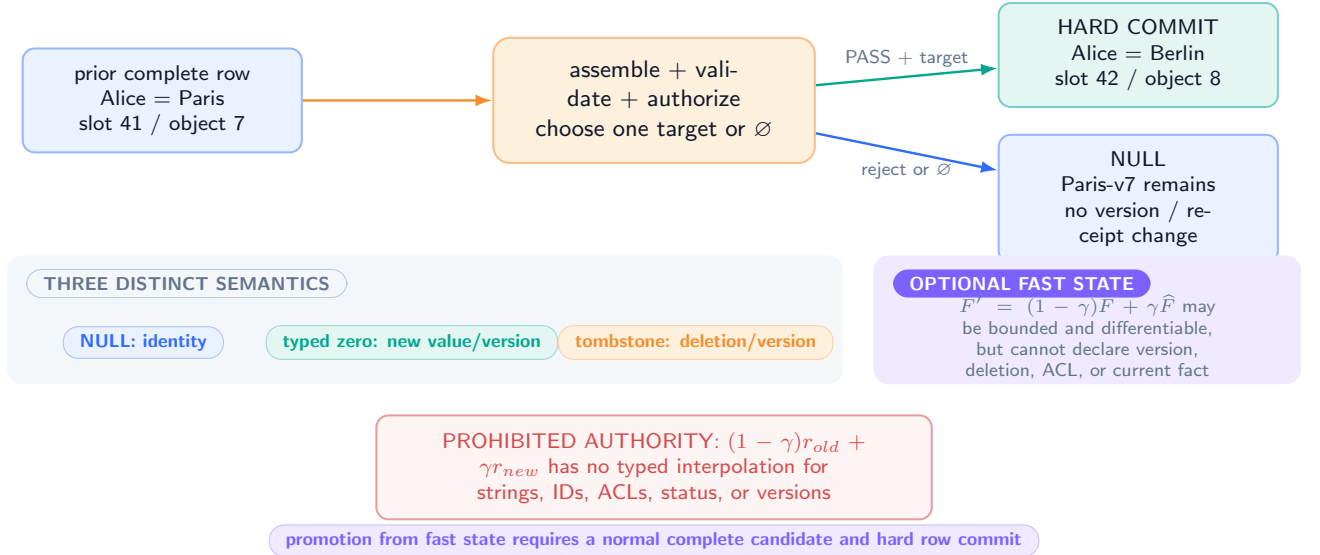
This does not prohibit differentiable computation. Let

$$F_t \in \mathbb{R}_q^{S_F \times d_F} \quad (41)$$

be a separately typed bounded fast state with its own lifetime and reset rule. Soft writes may update F_t ; generation may consume it as a heuristic. Conformance requires that F_t cannot declare a row live, deleted, protected, current, or authorized, and cannot bypass a current AMR read check. Promoting information from F_t to M requires a normal complete candidate and hard commit.

Authoritative outcomes: complete commit or exact NULL

typed zero and tombstone are commits; soft mixing belongs only to a separately declared fast-state type



The hard boundary is a state-meaning contract, not a ban on differentiable computation.

Figure 6: **Authority has two outcomes: a complete hard commit or exact NULL.** A typed zero is a committed value; a tombstone is a committed deletion state. Soft interpolation may exist only in a separately bounded, non-authoritative fast state and must pass ordinary assembly before promotion.

6.4 Zero, tombstone, and NULL

A typed canonical integer zero has a type and length; an all-zero byte string may be a valid value; and an all-zero quantized payload may be valid when its canonical relation passes. Therefore a legal complete-row replacement with a zero value is still a commit. It advances the slot sequence, normally advances object version, and creates a receipt:

$$\text{Replace}(M, a, \hat{r}_a^{\text{zero}}) \neq T_{\text{NULL}}(M). \quad (42)$$

A tombstone is likewise a complete new row with status, identity, versions, deletion authorization, and retention. Its active payload normal form may contain zeros, but the zeros do not define deletion.

6.5 Segment partial success

Equation 34 is prefix-committing. If event j commits and event $j + 1$ returns NULL, the earlier commit remains. The whole segment is not automatically all-or-nothing. A bounded segment transaction would require a separate transaction object that lists every row, read/write set, precondition, rollback record, byte bound, and atomic publication mechanism. Such a protocol is future work and cannot be inferred from per-event atomicity.

7 Lifecycle Semantics

Lifecycle names describe admissible old/new row pairs. They do not introduce extra physical operators.

7.1 Version rules

Every accepted replacement satisfies

$$v'_s = v_s + 1. \quad (43)$$

If logical object identity is unchanged,

$$v'_o = v_o + 1; \quad (44)$$

if a different object is inserted, $v'_o = 1$. A required increment is defined only below the maximum representable value. At exhaustion the event returns NULL with a typed failure and requires an external re-key or migration protocol. It never wraps, saturates, or silently reuses a prior (a, v_s) or (o, v_o) pair.

Protection classes form a finite preorder $(\mathcal{P}, \preceq)$. An ordinary event may preserve or strengthen protection, $\pi_{\text{old}} \preceq \pi_{\text{new}}$. Weakening requires an authenticated capability bound into provenance. Retention is distinct: it determines eligibility for later deletion/purge, whereas protection governs authorization. The passage of time alone performs no memory mutation.

7.2 Operation table

DEFINITION

Operation	Required old state	Required new-row semantics
Insert	EMPTY	LIVE; new object; object version 1; slot sequence +1
Update	LIVE	LIVE; same object; complete canonical/neural replacement; both versions advance
Consolidating update	LIVE	LIVE; same object; one complete canonical capsule may summarize several observed facts; no second resident row is invalidated
Evict-and-insert	eligible LIVE/TOMBSTONE	LIVE; different object; old object is not copied into new rollback lineage
Delete	LIVE	TOMBSTONE; same object; content unavailable; deletion authority, reason, retention, and new receipt
Alias rename	LIVE/TOMBSTONE	same object/status; bounded alias array replaced; version advances; index projection rebuilt
Protect / ACL change	LIVE/TOMBSTONE	same object/content; policy-authorized control change; complete row reauthenticated
Rollback	LIVE/TOMBSTONE/ QUARANTINED	prior same-object semantic content; current security policy; new forward versions
Quarantine	recoverably identified corrupt row	QUARANTINED; active content unavailable; bounded reason and recovery provenance
Repair	QUARANTINED	newly validated LIVE or TOMBSTONE row from an authenticated source
Purge	retention-eligible TOMBSTONE	EMPTY; slot sequence advances; external deletion/archive policy applied
Reject	any	NULL; every authoritative bit unchanged

Table 4: Lifecycle interpretations of one complete-row replacement. Preconditions are part of validation.

7.3 Insert, update, and consolidation

Insertion allocates a new object identity or validates a preallocated one, starts object version one, and binds the row to the target tenant and slot. An update preserves object identity and replaces the complete semantic state. The old canonical or neural bytes do not survive as an independently readable shadow.

A consolidating update may combine several observations into one canonical tuple when they describe one logical object. For example, a row may replace separate observed location and timestamp fields with one typed current-location tuple. The assembler must declare which source receipts contribute and must fit them into bounded provenance. Consolidation does not authorize deleting a second resident object. If two resident rows represent duplicates, retiring one and enriching the other takes two ordered events or a future bounded transaction.

7.4 Eviction

Eviction is not deletion of an object from all storage. It is replacement of an eligible resident slot with a different object. The policy must respect tenant partition, protection, retention, legal hold, and any required archive precondition. The displaced object may be recoverable from a separately charged archive, but it is not hidden inside the new occupant’s rollback ring. If no row is eligible, the event returns NULL.

The eviction target may be proposed by a learned ranker, but eligibility and final selection constraints are trusted. This paper does not validate a learned eviction policy.

7.5 Deletion and tombstones

A deletion request needs a capability distinct from ordinary write permission. A committed tombstone retains enough bounded identity, version, authorization, and retention information to suppress stale resurrection during its declared scope. It does not assert that source documents, backups, replicas, archives, or model parameters were erased.

An expired retention deadline does not mutate the row. A later authorized purge may replace the tombstone with EMPTY after archive, legal-hold, replica, and tenant policy checks. Once purged, perpetual non-resurrection over an unbounded object universe is impossible without an external authoritative deletion ledger; Theorem 8.15 makes this limit explicit.

7.6 Alias rename

An alias rename is a same-object complete replacement. The primary typed key remains authoritative; aliases are bounded query handles. Removing an alias means it disappears from the canonical capsule and therefore from the next deterministic index projection. An old alias-index entry contains the prior slot sequence and receipt digest and cannot be accepted after current-row revalidation.

Alias collision has two legal profile choices. A strict profile rejects the candidate. A permissive profile returns all current rows whose validated alias projections match and requires a later disambiguation step; it never makes “first index hit” authoritative.

7.7 Rollback moves content backward and lineage forward

Let snapshot s_j be a same-object entry in the bounded rollback ring. A legal rollback constructs

$$\begin{aligned} c' &= c(s_j), \\ n' &= \text{Conform}_{\sigma, \omega}(c'), \\ v'_s &= v_s + 1, \\ v'_o &= v_o + 1. \end{aligned} \tag{45}$$

Current ACL, tenant binding, retention, and protection are not blindly copied from the snapshot. Restoring old content is a compensating transaction, not time reversal. Rollback from a tombstone is an explicit resurrection and requires the corresponding authority.

PROVED UNDER STATED ASSUMPTIONS

Proposition 7.1 (Monotone rollback). *Under the rollback rule and authorization checks, a legal rollback may restore prior same-object semantic content but cannot decrease slot sequence, object version, or current protection without a separately authenticated weakening capability.*

Proof. The assembler reads semantic fields only from a same-object bounded snapshot. It computes both new versions from the current row by addition of one and rejects overflow. Current protection is inherited or strengthened unless χ carries a valid weakening capability. Therefore semantic content can revert while lineage moves forward. $\square$

Lifecycle meanings share one replacement operator and forward-only lineage

DEFINED TRANSITIONS / MONOTONE ROLLBACK PROVED

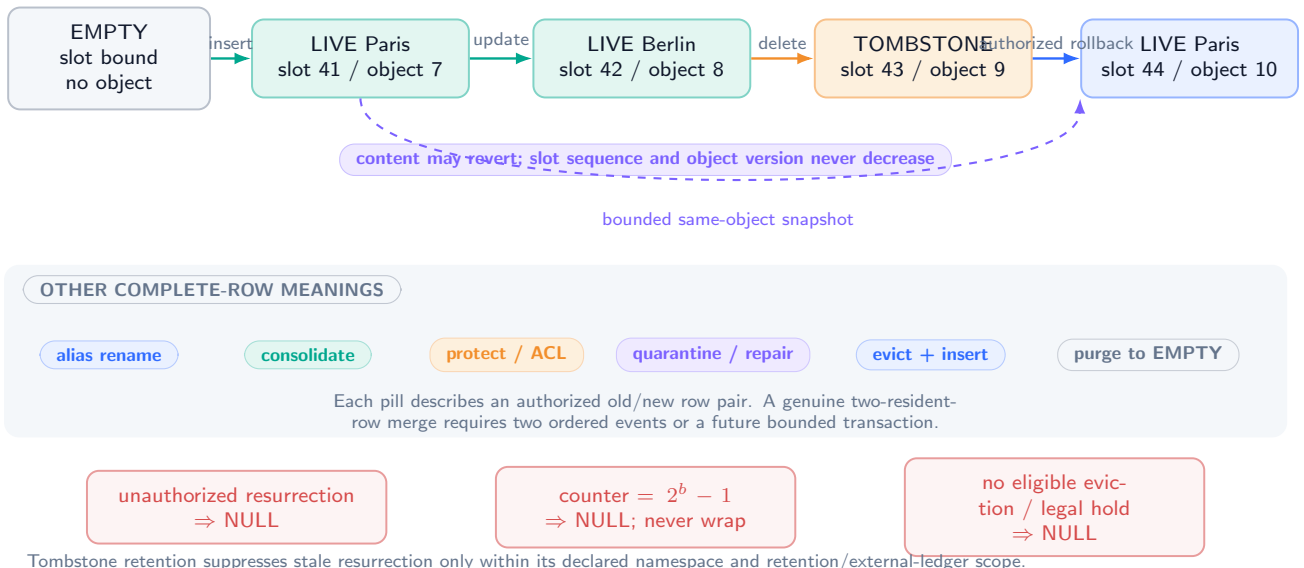


Figure 7: **Lifecycle states share one replacement operator and forward-only lineage.** Delete commits a tombstone; rollback restores bounded same-object content as a new version; purge is a later authorized event. The red paths are bounded rejection or overflow, not ordinary data flow.

7.8 Two-row merge limitation

PROVED UNDER STATED ASSUMPTIONS

Proposition 7.2 (One event cannot implement a genuine two-row merge). *Suppose a merge specification requires both (i) row a to contain a combined object and (ii) distinct resident row $b \neq a$ to cease being live in the same atomic outcome. No transition of Eq. 32 implements that specification.*

Proof. A non-NULL transition changes exactly one target row. If it changes a , row b is untouched and remains live; if it changes b , row a cannot receive the combined object. NULL changes neither. Thus the required two-row postcondition is outside the event transition set. $\square$

Two ordered events can express “write combined object, then tombstone duplicate,” but a crash or rejection between them leaves a visible prefix. A future transaction can make them all-or-nothing only by adding an explicitly bounded multi-row protocol. Calling one-row consolidation a merge does not remove this limitation.

8 Invariants and Preservation Results

8.1 Assumption ledger

The results are conditional. They do not assert that a deployed system satisfies these premises.

Assumption 8.1 (Valid genesis). The initial table M_0 contains exactly S parseable schema- σ rows, each in the appropriate empty normal form with valid tenant/slot bindings and receipts.

Assumption 8.2 (Trusted computing boundary). The assembler, authentication context, policy engine, key material, canonicalizer, and commit coordinator are not controlled by the neural proposer. Their code may still contain bugs; this assumption excludes adversarial replacement of the trusted functions in the formal model.

Assumption 8.3 (Deterministic interpretation). Serialization, parsing, canonicalization, quantization, the AMR-1 encoder, dual-view validation, policy evaluation, and base-index derivation are deterministic for their recorded schema/build identifiers.

Assumption 8.4 (Snapshot-consistent validation). Every local and global check is evaluated against the same table snapshot and authenticated context from which versions and predecessor digests are computed.

Assumption 8.5 (Serializable publication). Accepted events for one tenant are placed in a total order, or an equivalent tenant-epoch compare-and-swap rejects a stale snapshot. Global uniqueness checks and their publication share that order.

Assumption 8.6 (Atomic durable publication). A storage transaction exposes and, after recovery, returns either the complete old table root or the complete new table root, with the corresponding required audit-root binding. Derived indexes may lag and are rebuilt from validated rows.

Assumption 8.7 (Cryptography). H is collision resistant and the authentication scheme is unforgeable under chosen-message attack for parties without the relevant key. Key identifiers, rotations, and revocations are authenticated.

Assumption 8.8 (Static tenant partition). P does not change during ordinary event execution, and the read observation of tenant t projects only slots in $P^{-1}(t)$ after current-row authorization.

Assumption 8.9 (Logical observation model). Tenant noninterference excludes timing, cache occupancy, shared accelerator contention, global scheduling, shared-model memorization, allocator leakage, and malicious privileged administrators.

Assumption 8.10 (External-ledger contract). When policy requires an archive or audit record, publication occurs only after that record is durably prepared and its fixed digest/root is bound into the commit. If capacity or durability fails, the authoritative result is NULL.

8.2 Invariant family

Let $\text{Inv}_\sigma(M)$ be the conjunction of:

- I1. Fixed shape.** $|M| = S$, every row is B_R bytes, parses under σ , and has zero reserved bits.
- I2. Slot and tenant binding.** Row i declares slot i and tenant $P(i)$.
- I3. Status normal form.** EMPTY, LIVE, TOMBSTONE, and QUARANTINED obey the field constraints of Section 2.
- I4. Canonical bounds.** Type, length, alias, normalization, padding, and missing-value encodings are valid.
- I5. Neural finiteness.** Every coordinate is in the declared finite domain and forbidden encodings are absent.

- I6. Dual-view conformance.** Every readable LIVE row passes its declared mechanical relation and required validation bits.
- I7. Security integrity.** Tenant, owner, ACL, deletion authority, and protection fields were system assembled and parse uniquely.
- I8. Version discipline.** Every accepted predecessor relation follows Eqs. 43–44; no counter wraps.
- I9. Protection discipline.** Protection is not weakened without a bound authenticated capability.
- I10. Provenance completeness.** Every non-genesis row records the bounded event/source/build/operation fields required by its schema.
- I11. Receipt validity.** The core checksum, predecessor digest, validation bitmap, algorithm IDs, event binding, and authentication tag verify.
- I12. Rollback bounds.** Lineage has at most H_{rb} nonrecursive same-object snapshots and obeys current tenant scope.
- I13. Identity uniqueness.** Within a tenant, no two LIVE or retained TOMBSTONE rows claim the same nonzero object identifier or primary typed key.
- I14. Tombstone discipline.** A tombstone cannot become LIVE without explicit resurrection/rollback authority; purge obeys retention and legal hold.
- I15. Read discipline.** Only a current LIVE row passing receipt, conformance, tenant, ACL, validity, and protection checks may supply active content.
- I16. Derived-view discipline.** An index or cache can return only a hint that is revalidated against the current row; it cannot change authority.

8.3 Preservation

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.11 (Invariant preservation). *Under Assumptions 8.1–8.5, if $\text{Inv}_\sigma(M)$ holds and M' is produced by a legal event transition, then $\text{Inv}_\sigma(M')$ holds. Every finite sequence of legal transitions from genesis therefore preserves the invariant family.*

Proof. If the action is NULL, $M' = M$ and every conjunct is unchanged. Otherwise exactly target row a is replaced. For $i \neq a$, bytes are identical, so every row-local invariant persists. The trusted assembler fixes slot and tenant, normalizes and bounds canonical content, verifies finite neural encodings and dual-view conformance, applies status normal form, computes versions without overflow, enforces ACL/deletion/protection policy, constructs bounded same-object rollback, binds provenance, zeros reserved bytes, and authenticates the complete core. Thus I1–I12 and the row-local parts of I14–I16 hold for the candidate.

Identity/key uniqueness and any strict alias rule are evaluated against the same snapshot used for candidate assembly. Serializable publication or tenant-epoch compare-and-swap prevents a conflicting interleaving between the global check and publication. Thus I13 remains true. Tombstone/resurrection and read rules are explicit validation predicates, so I14–I15 hold. Derived views are projections and are not part of the authoritative replacement; every future use revalidates against the current row, preserving I16. Therefore all conjuncts hold in M' . Induction on the finite event sequence proves the second claim. $\square$

PROVED UNDER STATED ASSUMPTIONS

Corollary 8.12 (Version monotonicity). *Along every accepted predecessor chain for one physical slot, slot sequence is strictly increasing until exhaustion. Along a resident same-object chain, object version is strictly increasing. NULL leaves both unchanged.*

Proof. The assembler rules are invariant I8 and are preserved by Theorem 8.11. A commit applies +1 and rejects overflow; NULL is identity. $\square$

PROVED UNDER STATED ASSUMPTIONS

Corollary 8.13 (Untouched-row preservation). *For event target a , every row $i \neq a$ is byte-identical before and after the event. Across a segment, every slot outside the union of committed targets is byte-identical between M_{n-1} and M_n .*

Proof. The first statement is Eq. 30. Composition can change only a row selected by at least one event, yielding the second statement. $\square$

8.4 Finite versions and deletion history

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.14 (Finite monotone-version exhaustion). *A b_s -bit slot sequence initialized at zero and required to increase strictly on every commit permits at most $2^{b_s} - 1$ commits before the next commit must be rejected or the namespace must be migrated.*

Proof. There are 2^{b_s} representable values. A strictly increasing sequence starting at zero can visit each at most once. After $2^{b_s} - 1$, no larger value exists in the representation. $\square$

Silent wraparound would make stale receipts and current versions indistinguishable. An epoch-renewal protocol may extend lifetime, but its epoch is additional authoritative state and must itself be bounded, authenticated, and migrated.

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.15 (Bounded deletion-memory impossibility). *A system with B total authoritative bits cannot, without external authoritative state, distinguish forever all deletion histories over an unbounded object universe.*

Proof. Choose distinct objects $o_1, o_2, \dots$ and let M_j be the authoritative state after deleting the first j objects. Among $M_0, \dots, M_{2^B}$, two states coincide by the pigeonhole principle: $M_p = M_q$ for some $p < q$. Object o_{p+1} is deleted in the history producing M_q but not in the history producing M_p . A deterministic query over identical final authoritative states cannot answer differently. $\square$

The coherent guarantee is therefore scoped: a finite namespace, a bounded tombstone set, a retention window, or an external deletion ledger. “Permanent deletion memory for arbitrarily many identities at fixed total cost” is not a satisfiable specification.

8.5 Receipt tamper evidence

PROVED UNDER STATED ASSUMPTIONS

Theorem 8.16 (Receipt tamper evidence). *Under Assumption 8.7, an adversary without the authentication key who changes any receipt-covered core bit of a valid row can make the changed row verify only by finding a hash collision or forging the authentication scheme.*

Proof. If the altered core has the old digest, the adversary found a collision. If its digest changes, the old authentication tag no longer authenticates the receipt message and acceptance requires a valid tag on a new message, which is a forgery. $\square$

This theorem authenticates bytes and declared lineage. It does not authenticate real-world truth, protect a compromised key, or prove the correctness of the trusted code that created the receipt.

9 Security, Index, Concurrency, and Recovery Contract

9.1 Tenant and principal scope

For authenticated context χ with tenant t , the feasible targets satisfy

$$\mathcal{A}_t(M, \chi) \subseteq P^{-1}(t) \cup \{\emptyset\}. \quad (46)$$

The assembler additionally requires the candidate tenant to equal $P(a) = t$. A tenant cannot evict another tenant’s row to obtain capacity. Static quotas can waste space, but they make the logical isolation statement auditable.

Within a tenant, each ACL entry contains a principal identifier and rights mask. A reference set is

$$\{\text{READ, WRITE, DELETE, PROTECT, ROLLBACK, ACL_ADMIN}\}. \quad (47)$$

ACL changes require ACL_ADMIN; delete and rollback require their own rights. A general WRITE bit does not authorize security weakening or resurrection.

Formal noninterference traditionally asks whether one domain’s behavior can affect another domain’s observation [4]. Here the observation model is intentionally narrow.

PROVED UNDER STATED ASSUMPTIONS

Theorem 9.1 (Logical tenant noninterference). *Under Assumptions 8.5, 8.8, and 8.9, for $t \neq t'$ and a legal event issued under tenant t ,*

$$\text{Obs}_{t'}(T_t(M)) = \text{Obs}_{t'}(M). \quad (48)$$

Proof. Every feasible target of tenant t lies in $P^{-1}(t)$. Static partitions are disjoint, so no row in $P^{-1}(t')$ changes. The observation of t' projects only current authorized rows in its partition and rejects any index entry whose tenant/current-row binding does not match. Therefore its logical observation is unchanged. $\square$

This theorem excludes timing, shared caches and accelerators, global backpressure, traffic analysis, shared-model memorization, and malicious administrators. A production confidentiality claim needs a stronger observation model and empirical side-channel analysis.

9.2 Derived indexes and stale-value suppression

Every base index is a deterministic function $I = D_{\sigma}(M)$. An entry is a hint containing

$$(\text{tenant}, \text{object}, \text{key-digest}, a, v_s, d_p, \text{projection}). \quad (49)$$

A lookup accepts the hit only if the current row at a has matching tenant, object, slot sequence, readable status, and receipt digest, then passes the ordinary receipt, dual-view, ACL, validity, and protection checks.

PROVED UNDER STATED ASSUMPTIONS

Theorem 9.2 (Stale-index safety). *An arbitrarily stale derived index cannot make an obsolete, deleted, replaced, or cross-tenant row acceptable when every hit is revalidated by Eq. 49 and the current-row read predicate.*

Proof. If the row has not changed, validation reduces to the ordinary current read. If it has changed, then either the slot sequence, object identity, status, or receipt digest differs and the hit is rejected. A cross-tenant entry fails tenant/partition validation. A retained tombstone fails readable status. Thus a stale entry cannot authorize stale content. $\square$

A stale or missing index can cause a false negative, fallback scan, extra work, or denial of service. It is safe only in the authority sense, not necessarily available or efficient. AMR-1 updates nine canonical entries and one neural entry after a row commit, for exactly

$$9 \times 112 + 336 = 1,344 \text{ logical index bytes}. \quad (50)$$

If the update is lost, rebuild from validated rows restores the deterministic projection.

9.3 Serial execution and linearization

Database transactions provide the classical all-or-nothing abstraction, while linearizability relates concurrent operations to a legal real-time-respecting sequential history [7, 8]. AMR adopts a narrower one-row event contract.

DEFINITION

Definition 9.3 (Commit linearization point). The linearization point of an accepted event is the atomic publication of the new authenticated table root and required audit-root binding. A NULL event linearizes at the final validation decision and leaves the root unchanged.

PROVED UNDER STATED ASSUMPTIONS

Theorem 9.4 (Serializable event history). *Under snapshot-consistent validation and serializable publication, every finite concurrent execution of accepted events is observationally equivalent to a sequential execution ordered by their successful publication points, with rejected stale attempts represented as NULL.*

Proof. The commit coordinator assigns each successful publication a unique position in a tenant-total order (or a stronger global order where uniqueness spans tenants). A candidate whose snapshot epoch is no longer current fails compare-and-swap and produces no authoritative mutation. Consider successful publications in their assigned order. Each transition was validated against the predecessor root it replaces and changes one complete row; therefore applying those transitions sequentially yields exactly the published roots. Reads validate against one published root and can be placed after the root publication they observe. The resulting sequential history respects successful publication order and has the same observations. $\square$

The theorem would fail if two rows could pass a uniqueness check on different snapshots and publish independently. Per-row atomicity is therefore insufficient for memory-global uniqueness unless the uniqueness constraint has a serializable authority of its own.

9.4 Crash-safe publication protocol

ARIES and related recovery systems show that crash recovery needs an explicit log and ordering discipline rather than the word “atomic” alone [9]. The following is an abstract obligation, not an implementation claim.

Protocol 9.5 (Copy-on-write root publication). For current root R and candidate row:

1. allocate a new immutable row/page and any required audit record; compute their digests;
2. construct a new immutable table root R' pointing to the new row and binding the required audit-ledger sequence/root;
3. durably write and verify the new row, audit record, and R' in an order that prevents a root from referencing unprepared data;
4. atomically publish one checksummed root selector from R to R' ;
5. update derived indexes after publication or rebuild them later;

6. on recovery, select the latest root whose selector, table, rows, and required audit binding validate; otherwise retain the prior valid root.

The root selector, durability primitive, sector/page atomicity, flush/fence ordering, replica protocol, and failure model are deployment parameters. A write-ahead implementation may satisfy the same abstract old-or-new condition with different steps.

PROVED UNDER STATED ASSUMPTIONS

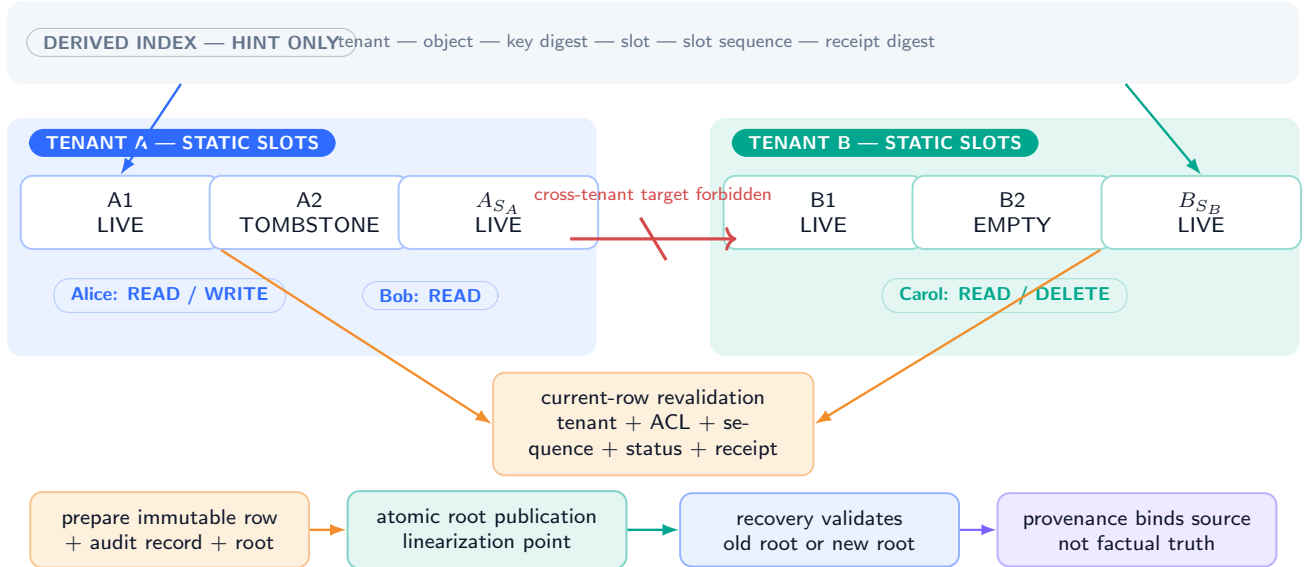
Theorem 9.6 (Old-or-new crash recovery). *Assume Protocol 9.5 is implemented correctly; durable writes do not acknowledge before reaching the stated failure domain; the root selector update is atomic; and recovery accepts only fully validated roots. After any single crash during one commit, recovery exposes either the complete precommit table and audit binding or the complete postcommit table and audit binding, never a readable mixture.*

Proof. Before the selector publication, the old selector still names R , whose referenced objects remain immutable and valid; unreferenced prepared objects are garbage after recovery. After selector publication, the protocol guarantees that every object reachable from R' was durably prepared. If the selector itself is torn or invalid, its checksum/redundancy causes recovery to choose the prior valid selector; if it is valid, recovery validates all reachable objects before accepting R' . Thus the recovered root is R or R' . Because a row is immutable and both views are inside it, neither root contains a torn canonical/neural or control/content combination. $\square$

UNVALIDATED No storage engine, filesystem, device, replica set, flush sequence, or crash injector is evaluated in this paper. The theorem describes what an implementation must establish.

Tenant isolation, provenance, derived indexes, and crash-safe publication

LOGICAL RESULTS PROVED UNDER EXPLICIT TRUST / STORAGE ASSUMPTIONS



Not covered: timing/cache/accelerator side channels, shared-model memorization, compromised trusted code or keys, and external deletion.

Figure 8: **Static tenant partitions, system-owned security, and current-row revalidation define the logical boundary.** Indexes are hints, provenance binds declared sources, and publication binds the row to an audit root. The proved path excludes side channels and depends on authenticated context, serializable checks, and crash-safe storage assumptions.

9.5 Threat model

The model includes malicious or erroneous neural proposals, replayed stale candidates, malformed serialized fields, alias collisions, cross-tenant target attempts, unauthorized deletion or rollback, storage bit corruption, stale indexes, interrupted commits, and parties without receipt keys attempting row tampering.

It does not defeat a compromised trusted assembler, policy engine, authentication service, signing/MAC key, storage administrator, firmware, or validator build. It does not cover covert channels, denial of service through shared compute, model-parametric memorization, source truth, or complete erasure from backups. The system can fail closed under many of those conditions, but availability and recovery may then require external administration.

10 Complete Cost Calculus

10.1 Why one scalar hides the contract

Candidate construction mixes neural inference, byte traffic, comparisons, policy checks, cryptography, storage barriers, and index work. We therefore report a work vector

$$\mathcal{W} = (B_{rd}, B_{wr}, N_H, N_{authV}, N_{authC}, N_E, N_D, N_{pol}, N_{cmp}, N_{barrier}), \quad (51)$$

where the coordinates count logical bytes read/written, hash compression blocks, authentication verifications/creations, encoder and dual-view validator calls, policy evaluations, identity/key comparisons, and durable ordering barriers. Model FLOPs, accelerator traffic, kernel launches, wall time, replication, and energy are additional measured coordinates, not constants hidden in $O(1)$.

For a complete deployment, define:

- C_{event} : bounded segment eventization cost;
- C_G : neural proposal cost per target-conditioned candidate;
- C_A : trusted normalization, assembly, and local validation;
- C_D : dual-view derivation/certification;
- C_R : target routing or shortlist construction, including misses and excluded actions;
- C_U : memory-global uniqueness/protection checks;
- C_Q : action scoring after candidates exist;
- C_P : durable row/audit publication;
- C_I : index projection, update, rebuild, and lookup validation;
- C_X : archive write, lookup, retrieval, retention, and deletion work; and
- C_{audit} : audit append/checkpoint, authentication, and retention work.

No end-to-end bound may report only row-read cost while omitting C_G , C_A , or the candidate multiplicity used by routing.

10.2 General candidate accounting

Let B_P be proposal bytes, B_C canonical bytes scanned, B_D neural route/payload bytes compared, $B_{core} = B_R - B_P$, S_t tenant slots, B_{ctrl} trusted-control bytes scanned per row, and B_{key} primary-key bytes scanned per row. A conservative exact logical safe path that reads the old row, reads the proposal, normalizes canonical bytes, materializes and compares the neural view, hashes the complete core, and scans the tenant for object/key uniqueness has

$$\begin{aligned} B_{rd}^{cand} &= B_R + B_P + B_C + 2B_D + B_{core} \\ &\quad + S_t(B_{ctrl} + B_{key}), \\ B_{wr}^{cand} &= B_R + B_D. \end{aligned} \quad (52)$$

This is protocol traffic, not a prediction of cache misses: an implementation may fuse passes or reuse a cache line, but must report the measured physical traffic separately.

For a Merkle–Damgård-style hash whose compression block is r_H bytes, domain prefix is B_{dom} bytes, and padding consumes one delimiter plus an eight-byte length field, the receipt core needs

$$N_H = \left\lceil \frac{B_{dom} + B_{core} + 9}{r_H} \right\rceil \quad (53)$$

compression blocks. SHA-256’s block structure supplies the illustrative AMR-1 numbers [11]; another profile must recalculate them.

One candidate has abstract cost

$$\begin{aligned} C_{cand} &= C_G + C_{normalize}(B_C) + C_D + C_{pol} + C_U \\ &\quad + N_H C_{H-block} + C_{source-auth} + C_{receipt-auth} \\ &\quad + C_{traffic}(B_{rd}^{cand}, B_{wr}^{cand}). \end{aligned} \quad (54)$$

Rejecting after step q costs the prefix actually executed. A schema-length rejection can occur before encoder and global scans; a late duplicate-key or authentication failure can incur almost the entire candidate cost while still producing zero authoritative row bytes.

10.3 AMR-1 candidate path

For AMR-1,

$$\begin{aligned} B_R &= 6,592, & B_P &= 1,888, & B_C &= 1,104, & B_D &= 768, \\ B_{\text{core}} &= 6,432, & B_{\text{ctrl}} + B_{\text{key}} &= 256 + 66 = 322. \end{aligned} \quad (55)$$

The proposal is exactly the 1,104-byte canonical block plus 784-byte neural block. The dual comparison covers only the 768 route/payload bytes; the 16 neural metadata/reserved bytes are separately validated.

With $S_t = 256$, the authoritative uniqueness scan reads

$$256 \times 322 = 82,432 \text{ bytes.} \quad (56)$$

The remaining local reads are

$$6,592 + 1,888 + 1,104 + 2(768) + 6,432 = 17,552 \text{ bytes,} \quad (57)$$

so

$$B_{\text{rd}}^{\text{cand}} = 99,984 \text{ bytes.} \quad (58)$$

Materializing the complete candidate and one 768-byte encoder scratch buffer writes

$$B_{\text{wr}}^{\text{cand}} = 6,592 + 768 = 7,360 \text{ bytes.} \quad (59)$$

With a 16-byte domain prefix and 64-byte SHA-256 compression blocks,

$$N_H = \left\lceil \frac{16 + 6,432 + 9}{64} \right\rceil = 101. \quad (60)$$

The reference operation ledger is therefore one neural proposal, one canonical normalization, one deterministic encoder call, one policy evaluation, 256 object/key comparisons, 101 hash compression blocks, one source-authentication verification, one receipt-authentication creation, 99,984 specified precommit read bytes, and 7,360 specified precommit write bytes. A strict alias-uniqueness scan adds

$$256 \times 524 = 134,144 \text{ read bytes} \quad (61)$$

and at most 8×256 alias comparisons.

On rejection, authoritative publication bytes are zero. On acceptance, logical publication adds one 6,592-byte row and one 1,344-byte fixed index projection. The row may be prepared and written more than once physically under journaling, copy-on-write, or replication; that amplification is measured rather than inferred.

10.4 Routing and candidate multiplicity

If event j constructs $N_{n,j}$ candidates, its construction/validation cost is

$$\sum_{a \in C_{n,j}} C_{\text{cand}}(a), \quad |C_{n,j}| = N_{n,j}. \quad (62)$$

Exhaustive target conditioning gives $N_{n,j} = S_t$. A shortlist of size L may reduce this term only if C_R , index build/update/storage, shortlist misses, and omitted feasible targets are reported. Scoring S_t cheap target IDs while constructing only the winning row is not equivalent to the complete candidate set assumed by a controller whose action values depend on target-conditioned content.

For one segment,

$$C_{\text{segment}} = C_{\text{event}} + \sum_{j=1}^{K_n} \left(C_R + \sum_{a \in C_{n,j}} C_{\text{cand}}(a) + C_Q + C_{\text{selected publication}} \right). \quad (63)$$

The exact authoritative row traffic is $C_n B_R$ bytes. It is not $K_n B_R$ when events return NULL, and it does not count candidate buffers that never publish.

10.5 Re-encoding, archive, index, and audit costs

A refresh or re-encoding of N_{refresh} resident rows is N_{refresh} ordinary versioned candidates and commits. Its cost includes reading canonical content, running the new encoder, validating semantic gates, advancing versions, updating rollback, writing rows, and rebuilding index projections. There is no free in-place vector migration.

Let a commit emit B_A^+ archive bytes and B_L^+ audit bytes. Its full durable write volume includes at least

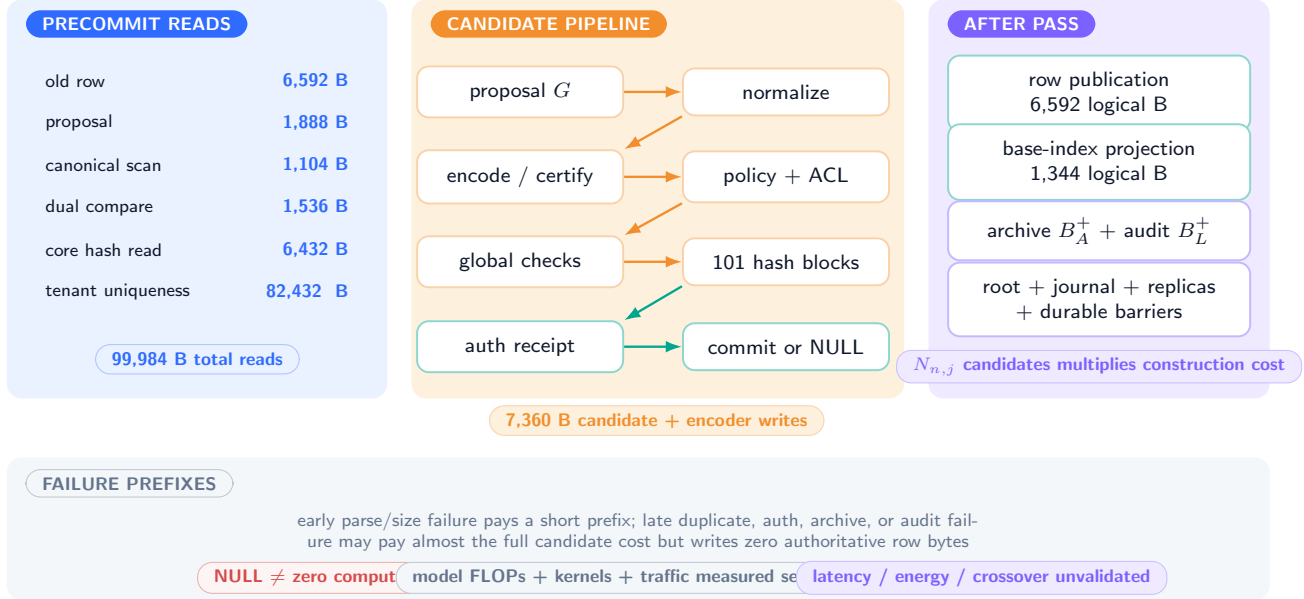
$$B_{\text{durable}}^+ = B_R + B_A^+ + B_L^+ + B_{\text{root}}^+ + B_{\text{replication}}^+ + B_{\text{journal}}^+, \quad (64)$$

plus ordering barriers and verification reads. Terms may be zero only when the profile explicitly does not require the corresponding artifact. Archive retrieval later incurs index lookup, storage reads, authentication, decompression/decoding, and candidate reconstruction; it cannot be counted only at write time.

An audit ledger with maximum $L_{\max}$ entries of fixed receipt size B_ℓ has capacity $L_{\max}B_\ell$ plus its bounded index/checkpoint. When full, the system fails writes whose contract requires audit, or executes a separately specified authenticated rotation that commits a checkpoint root before releasing old detail. An append-only ledger with no $L_{\max}$ is explicitly unbounded total storage.

Complete candidate and commit cost: exact logical ledger, explicit external terms

AMR-1 EXACT SAFE PATH / PHYSICAL PERFORMANCE UNVALIDATED



A shortlist is cheaper only after its construction, storage, miss, and excluded-action costs are charged.

Figure 9: **A complete candidate and commit have a countable bill.** The AMR-1 waterfall gives exact logical safe-path traffic for one candidate. Routing multiplicity, global checks, publication, index projection, archive, audit, replication, barriers, model FLOPs, latency, and energy remain visible rather than disappearing into a row-count asymptotic.

10.6 Failure-path ledger

Failure	Latest required work	Authoritative writes	Remaining cost/risk
Parse/size/type failure	proposal read + bounded parse	0	diagnostic write if enabled
Tenant/ACL/protection denial	old control + policy/auth checks	0	possible authentication and rate-limit cost
Dual-view failure	normalization + encoder/validator	0	optional bounded quarantine copy
Duplicate identity/key	tenant scan or validated uniqueness index	0	nearly complete candidate cost
Version overflow/stale snapshot	version/snapshot check; may be early or late	0	retry/re-key/migration external
Archive/audit capacity failure	archive/audit reservation and policy checks	0	cleanup of unreferenced prepared data
Crash before root publication	durable preparation may have occurred	0 visible	recovery/garbage collection
Crash after root publication	full commit durable by assumption	one row visible	index rebuild may remain
Stale index	lookup + current-row mismatch	0	fallback scan or false negative
Receipt mismatch on read	row read + cryptographic validation	0 by read	fail-closed unavailability and recovery

Table 5: Failure costs. NULL means zero authoritative mutation, not zero computation or zero external logging.

UNVALIDATED AMR-1's byte ledger is exact by construction. It does not predict cache behavior, latency, throughput, energy, device endurance, or quality–cost crossover. Those require an implementation and the stop-ruled measurements of Section 12.

11 Counterexamples and Failure Taxonomy

The contract’s assumptions are not decorative. Each prevents a concrete violation.

11.1 Unbounded aliases and provenance defeat boundedness

Counterexample 11.1 (Hidden variable-width row). Suppose a row stores a list $A(r)$ of every spelling ever used for its object, with no alias-count or length cap. After the m th rename, append a fresh one-byte-distinct alias. Then $|A(r)| \geq m$ and row storage grows with history even when $S = 1$. Likewise, storing the complete provenance document or all ancestors recursively makes B_p or B_λ history dependent. The statement “one fixed slot” no longer implies a bit bound.

Using a fixed digest pointer repairs resident boundedness only by moving the history to an external object. The total system remains unbounded unless that archive is independently capped.

11.2 Neural ownership of trusted fields breaks isolation and lineage

Counterexample 11.2 (Self-issued authority). Let the assembler copy proposed tenant, ACL, and version bits without independent derivation. A malicious proposer targeting a tenant-A slot can emit tenant B, grant its principal READ/DELETE, or reset version from 42 to 1. The serialized row may be well formed and its neural/canonical views may agree, yet tenant isolation, authorization, monotonicity, stale suppression, and rollback lineage all fail. A checksum computed over these bytes merely authenticates the bad assignment.

The remedy is functional system ownership: trusted fields are computed from authenticated context, current state, and policy, never self-certified by the proposal.

11.3 In-place neural refresh creates dual-view drift

Counterexample 11.3 (Two independently mutable truths). At version u , canonical bytes encode Paris and neural bytes equal $E_\omega(\text{Paris})$. A background process upgrades to ω' and overwrites only the neural region with $E_{\omega'}(\text{Berlin})$ or even $E_{\omega'}(\text{Paris})$ while leaving the receipt and version unchanged. The canonical and neural views no longer share one declared transformation and version. A canonical read and neural read can disagree, or stale cached validation can accept bytes produced by an undeclared model.

Re-encoding must therefore be a complete new versioned event, and reads/cache entries must be keyed by the current receipt.

11.4 Soft authoritative overwrite has no unique discrete lineage

Counterexample 11.4 (Versioned mixture ambiguity). Let a payload for Paris be v_P , a candidate for Berlin be v_B , and set $v' = (1 - \gamma)v_P + \gamma v_B$ for $0 < \gamma < 1$. If the canonical row remains Paris, the neural view no longer conforms to it. If canonical content becomes Berlin, the payload is not the declared Berlin encoding. If both canonical strings or ACLs are “mixed,” their interpolation is undefined. Assigning version 8 does not resolve which fact or security policy is authoritative.

Soft state is permitted only under the separate type and non-authority rule of Eq. 41.

11.5 Multi-event conflation breaks atomic claims

Counterexample 11.5 (Segment rank-one and merge error). A segment contains two events: update Alice in row 3 and update Bob in row 9. Both commits succeed with linearly independent payload changes. The segment difference has two nonzero rows and can have rank two. Calling the entire segment “one atomic overwrite” hides a partial-success boundary. Similarly, merging row 3 into row 9 while making row 3 non-live requires two resident changes and cannot be one event.

The correct recursion is Eq. 34; per-event support is at most one and segment support is at most K_n .

11.6 Per-row CAS does not preserve global uniqueness

Counterexample 11.6 (Write skew). Two assemblers read the same snapshot in which typed key k is absent. One prepares slot 7 and one prepares slot 8. Each uses a successful compare-and-swap only on its target row. Both commits can publish, leaving two live rows with key k . Every local row invariant holds; the memory-global uniqueness invariant fails.

A tenant epoch, serializable uniqueness index, or equivalent global commit check is required. This counterexample explains why the serializability assumption is stronger than atomic row bytes.

11.7 Failure taxonomy

Class	Example	Required authoritative behavior	Unresolved consequence
Syntax/bound	bad length, reserved bit, NaN, alias overflow	precommit NULL	proposal cost, possible denial of service
Authentication	bad tenant, ACL, capability, source tag	precommit NULL	compromised identity service out of model
Policy	protection/retention/legal-hold violation	precommit NULL	policy correctness not proven
Semantic consistency	canonical/neural mismatch or UNKNOWN	candidate NULL; resident read $\perp$	semantic validator errors unmeasured
Concurrency	stale epoch, uniqueness conflict	losing attempt NULL or retry	starvation/availability unproven
Storage integrity	checksum/tag mismatch, torn page	fail closed; recover old/new root	recovery depends on implementation
Index	stale/missing/corrupt hint	reject hit; scan or rebuild	latency and availability degradation
Version/history	counter exhaustion, rollback ring full	reject/migrate or evict declared snapshot	finite lifetime/history loss
Capacity	no eligible slot, audit/archive full	NULL unless explicit bounded rotation/eviction succeeds	memory stagnation or data loss trade-off
Adversarial truth	authenticated false source, poisoning	may still pass mechanical contract	truth/admission problem outside substrate
Key/build compromise	forged tag, malicious validator/assembler	outside theorem; revoke/recover administratively	rows may be untrustworthy
Side channel	timing/cache/traffic leakage	outside logical noninterference	separate security proof/test required

Table 6: Failure taxonomy. Fail-closed behavior preserves authority but can sacrifice availability.

11.8 Abuse cases and non-guarantees

Poisoning. An authenticated source can state a plausible falsehood whose canonical and neural views agree. Provenance enables attribution and later policy; it does not prove truth.

Capacity denial. An attacker can fill its tenant partition with low-value rows or retained tombstones. Static partitioning prevents cross-tenant eviction but not self-denial. Per-principal rate limits and protected reserve capacity are additional policies.

Rollback abuse. A valid old snapshot can still be obsolete or harmful. Rollback requires current authorization, tombstone-aware resurrection checks, and uniqueness validation; the substrate does not decide whether restoration is desirable.

Key or validator compromise. Receipt integrity ends at the trusted key and build boundary. Rotation and revocation are explicit migrations; receipts make a build attributable, not incorruptible.

Deletion scope. A row tombstone says nothing by itself about source documents, replicas, backups, archives, external logs, or learned model parameters. Each carrier needs its own deletion contract.

Protection deadlock. A policy can make all targets infeasible. NULL is safe for authority but can freeze useful learning. Administrative recovery needs its own bounded, authenticated, and audited path.

12 Falsification Protocols and Stop Rules

No protocol in this section has been executed for this manuscript. Passing a lower gate authorizes only the next prespecified test; it does not establish an end-to-end claim.

12.1 Contract-mechanics gates

PLANNED TEST

Protocol 12.1 (Finite-state model checking). Instantiate a reduced schema with small slot, version, ACL, status, alias, and rollback spaces. Exhaustively enumerate legal transitions, concurrent schedules permitted by the model, and recovery points.

Pass: no legal trace violates I1–I16 and every illegal trace is rejected. **Stop:** any legal trace reaches an invalid state; repair the specification before implementation claims.

PLANNED TEST

Protocol 12.2 (Cross-language serialization). Implement the same schema independently in at least two languages. Generate valid and invalid boundary rows including Unicode normalization, maximum lengths, typed zeros, counter maxima, reserved-bit corruption, alias collisions, NaNs, and every status normal form. **Pass:** byte-identical serialization, checksum/tag input, parsing, and accept/reject decisions. **Stop:** any equivalent input yields different authoritative bytes or validator decision.

PLANNED TEST

Protocol 12.3 (Transition and segment property tests). Generate segments with $K_n \in [0, K_{\max}]$, repeated targets, all-NULL segments, and mixed accept/reject outcomes. **Pass:** per-event row support ≤ 1 , $C_n \leq K_n$, $D_n \leq \min(C_n, S)$, untouched rows

remain byte-identical, and published row traffic is exactly $C_n B_R$. **Stop:** any hidden second row or segment-rank-one assertion appears.

PLANNED TEST

Protocol 12.4 (Version, tombstone, and rollback traces). Generate long randomized sequences of insert, update, alias rename, delete, rollback, protect, repair, purge, stale replay, and counter exhaustion. **Pass:** versions never decrease/wrap; rollback restores only bounded same-object content under current security; stale/unauthorized resurrection is zero during the declared scope. **Stop:** one unauthorized resurrection, version reuse, or obsolete ACL restoration.

12.2 Security and recovery gates

PLANNED TEST

Protocol 12.5 (Tenant and ACL adversary). Run concurrent requests over disjoint partitions with forged slot IDs, tenant IDs, object IDs, stale index entries, ACL mutations, delete requests, and rollback requests. **Pass:** zero cross-tenant row changes and readable content; zero security weakening without capability. **Stop:** any logical cross-tenant effect. Timing and shared-resource leakage are measured in a later, stronger model.

PLANNED TEST

Protocol 12.6 (Receipt and dual-view corruption). Flip every covered bit, substitute canonical/neural blocks and encoder IDs, replay predecessor receipts, alter validation bits, and corrupt cached verification keys. **Pass:** every unauthenticated alteration is rejected subject to the declared cryptographic model, and no mismatch yields a readable row. **Stop:** one changed covered core is accepted without acknowledged key compromise or cryptographic break.

PLANNED TEST

Protocol 12.7 (Crash injection). Implement one storage profile, then inject process, kernel, and device failures at every durable-write and root-publication boundary, including torn-selector simulations and index lag. **Pass:** recovery exposes exactly the old or new complete table/audit binding and stale indexes never authorize old content. **Stop:** any mixed canonical/neural, control/content, tenant/object, or row/audit state becomes readable.

PLANNED TEST

Protocol 12.8 (Index determinism and stale suppression). Rebuild indexes under different processing orders, drop updates, insert stale entries, and reuse slots for new objects. **Pass:** base rebuilds are byte identical; stale entries never authorize obsolete/deleted/cross-tenant rows; a full rebuild recovers the declared index. **Stop:** index state becomes independent authority.

12.3 Accounting gates

PLANNED TEST

Protocol 12.9 (Exact protocol-cost audit). Instrument every AMR-1 pass and compare proposal bytes, row scans, neural comparison, hash blocks, authentication calls, candidate buffers, row/index writes, archive/audit bytes, barriers, and failure-prefix work with Section 10. **Pass:** exact equality at the logical protocol layer and an explicit reconciliation to physical traffic. **Stop:** any mandatory component is unmeasured or hidden in a constant.

PLANNED TEST

Protocol 12.10 (Capacity and retention exhaustion). Fill rows, aliases, rollback rings, quarantine, optional indexes, archives, audit ledgers, tombstones, and counters to every boundary. **Pass:** the declared reject/evict/rotate/purge policy occurs with no silent allocation or authority spill. **Stop:** resident or total-system boundedness is claimed without reproducing the carrier inventory.

12.4 Semantic and reasoning gates

PLANNED TEST

Protocol 12.11 (Same-checkpoint semantic qualification). Compare canonical span, typed tuple, derived dual-view, certified dual-view, and pure latent control under one frozen checkpoint, fixed exposure, matched active parameters, fresh seeds, final-only evaluation, and truth-free materialization before scoring. Require canonical reconstruction, neural readout, fresh composition, query accuracy, calibration, old-value suppression, unrelated-row preservation, routing consistency, and corruption rejection at every fixed seed. **Stop:** if no interface passes all preregistered gates, no learned admission or reasoning integration is opened.

This gate must retain the accepted MMLA boundary: PM-I2’s 0/9 qualification and PM-I3’s fitted learned-content failure motivate a new interface test; they do not count as a test of AMR-1.

PLANNED TEST

Protocol 12.12 (Repeated-write reasoning integration). Only after substrate, crash, security, accounting, and semantic gates pass, compare an AMR-backed system with matched append-only context, canonical-only rows, and frozen retrieval. Measure factual update accuracy, stale leakage, untouched-fact preservation, unauthorized writes, tombstone/rollback correctness, answer quality, candidate multiplicity, total storage/index/archive/audit cost, latency, memory traffic, and energy. **Stop:** any reasoning gain accompanied by worse preregistered safety/integrity gates, or any gain that disappears under total-cost matching, blocks a system advantage claim.

12.5 Dependency and qualification graph

From broken contracts to conditional proofs to empirical qualification

counterexamples motivate premises; premises prove mechanics; separate tests are still required for implementation and usefulness

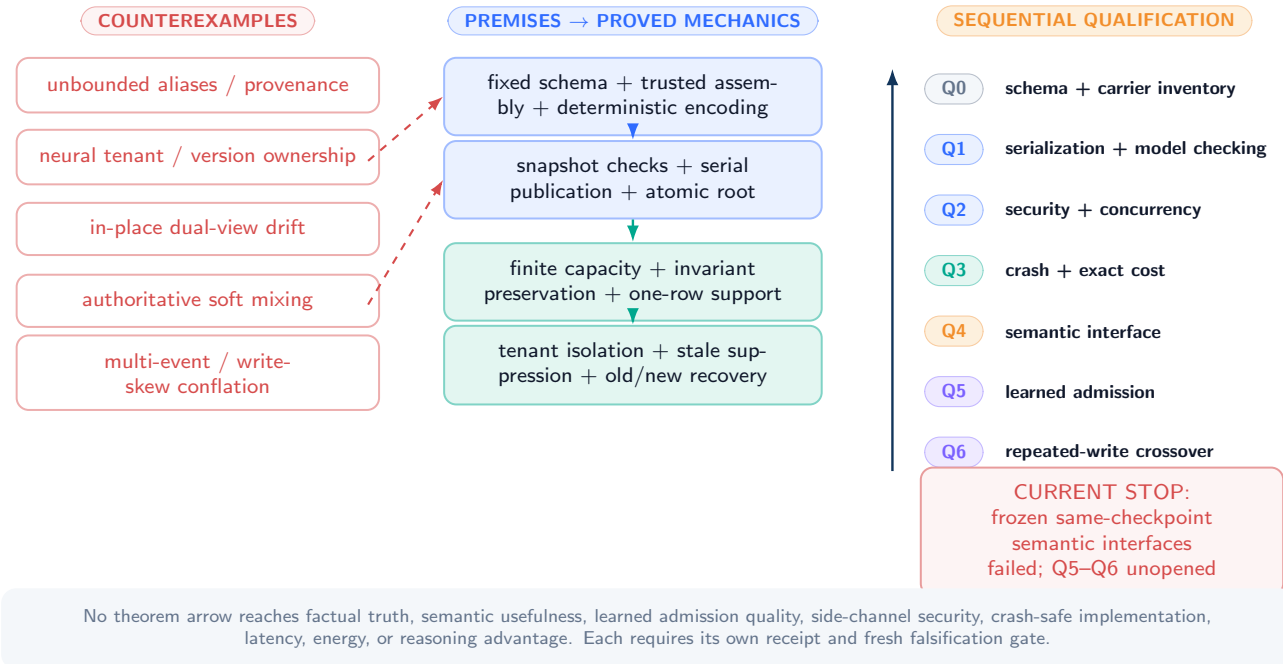


Figure 10: Counterexamples motivate premise bundles; premises support conditional theorems; theorems create implementation obligations, not empirical success. The qualification ladder is sequential. The frozen semantic interface remains failed, so learned admission and reasoning-integration rungs are unopened.

The complete theorem-to-obligation matrix is in Appendix C. Its central rule is simple: an implementation may cite a theorem only after producing evidence for every premise used by that theorem. A successful checksum test cannot substitute for atomic storage; a successful crash test cannot substitute for semantic qualification; and a semantically useful reader cannot substitute for tenant or deletion authorization.

13 Related Work, Limitations, and Conclusion

13.1 Neural external memory

Memory Networks and Neural Turing Machines established influential differentiable read/write stores, while the Differentiable Neural Computer added structured access mechanisms and broad task demonstrations [12, 5, 6]. Their central contributions concern learnable addressing, differentiability, and task performance. AMR asks a complementary systems question: which exact finite bytes may become authoritative when canonical and neural representations, security, provenance, deletion, and rollback must agree?

The distinction allows coexistence. A model may maintain soft differentiable scratch state for computation and use hard AMR commits only for resident authority. This paper does not claim that hard rows should replace every neural memory mechanism or that the reference dimensions are optimal.

13.2 Rows, transactions, and recovery

Atomic state transformation, transaction boundaries, serialization, and recovery are classical database problems. Gray articulated the transaction concept and its limits; ARIES developed write-ahead recovery with fine-grained locking and partial rollback; linearizability gives a concurrent-object correctness condition; and Bigtable is a prominent row-oriented storage system [7, 9, 8, 3]. AMR does not claim invention of rows, atomicity, versions, tombstones, logs, or recovery.

Its narrower contribution is a reasoning-state contract in which canonical content, a neural payload/routing key, identity, security, provenance, deletion, rollback, and a consistency receipt share one fixed-width authoritative boundary. Its event transaction is intentionally weaker than a general database transaction: one event changes at most one resident row, while global uniqueness and crash recovery require explicit coordinator/storage assumptions.

13.3 Provenance, deterministic encoding, and isolation

Database provenance distinguishes source and transformation histories [2]. AMR-1 embeds only a bounded receipt and uses an external ledger for richer history. Deterministic encoding is essential for cross-implementation checksums and signatures; RFC 8949’s deterministic CBOR discussion supplies useful precedent, while the profile here is stricter because each field has a fixed region [1]. SHA-256 and HMAC supply illustrative standardized digest and authentication profiles, not a mandate that every deployment use them [11, 10].

The tenant theorem follows the formal noninterference tradition but proves only equality of logical row observations under static disjoint partitions [4]. It does not claim constant-time execution, shared-hardware isolation, or protection from a privileged administrator.

13.4 Relationship to MMLA and the paper family

Public MMLA V3 is the historical source of the proposed complete-row, trusted-assembly, and one-overwrite-or-NULL boundary [13]. This manuscript turns that proposal into a standalone fixed-bit state machine, reconciled concrete profile, proof set, security/recovery assumptions, counterexamples, and complete cost ledger. It does not import V3’s controlled lifecycle or relational-binding experiments as validation of AMR-1. Those components were tested under other representations and boundaries.

The five-paper family separates concerns. The first focus paper defines policy-state witnesses for Reasoning-Time Training; this paper defines memory-state authority; the remaining papers formalize predictive admission, dual policy/memory state, and completed-segment consolidation. These are conceptual dependencies only, and each mechanism retains an independent evidence burden.

13.5 Limitations

1. The trusted computing base is substantial: authentication, assembler, policy, canonicalizer, encoder/validator, key management, commit coordinator, and recovery logic.
2. Static tenant partitions can waste capacity and do not address timing or shared-resource leakage.
3. The AMR-1 uniqueness scan is linear in tenant slots; a faster index adds its own authority and concurrency obligations unless treated only as a hint.
4. A deterministic encoder can be reproducible and still semantically useless. A learned validator can share the reader’s blind spots.
5. Fixed rollback forgets old history. A different evicted object requires an external archive or new observation.
6. Fixed-width versions eventually exhaust. Epoch renewal adds another bounded authenticated namespace.
7. Tombstones cannot remember an unbounded deletion set forever at fixed total cost.
8. Per-event atomicity is not segment-wide or multi-object transactionality.
9. Fail-closed validation protects authority at the cost of availability; corruption, key loss, or missing build artifacts can make rows unreadable.
10. A receipt binds bytes and declared checks, not truth, source reliability, validator competence, legal compliance, or ethical desirability.
11. Archive, audit, replication, filesystem, and backup costs are separate. Bounded hot state is not bounded total system storage.
12. Exact logical byte counts do not predict physical traffic, latency, energy, throughput, endurance, or hardware advantage.
13. No implementation, model checking, crash injection, security test, semantic qualification, or reasoning experiment is reported.

13.6 Explicit non-claims

This paper does not claim that:

- AMR-1 has been implemented or standardized;
- its 6,592-byte layout or dimensions are optimal;
- the public or technical-report evidence record validated a dual-view row;
- canonical/neural agreement establishes factual truth or semantic usefulness;
- checksums and receipts make a compromised trusted component safe;
- bounded rows improve language-model accuracy or reasoning;
- one-row atomicity implements general distributed transactions;
- tombstones guarantee global permanent deletion;
- rollback is always beneficial;
- static partitions eliminate all information leakage;
- deterministic indexes are fast or approximate indexes are safe without current-row revalidation;
- the profile lowers energy or wins a quality–cost comparison;
- memory-state mutation is itself strict policy training; or
- a theorem authorizes bypassing an empirical gate.

13.7 Conclusion

An Atomic Memory Row gives one resident object one typed identity, one canonical view, one bounded neural view, one tenant/security scope, one version lineage, one provenance receipt, and one publication boundary. The legal event is deliberately narrow: replace one complete row or perform bit-identical NULL.

Under explicit assumptions, that narrow rule yields useful formal consequences: fixed resident information capacity, one-row event locality and the correct K_n segment bound, invariant preservation, untouched-row preservation, monotone rollback, logical tenant isolation, stale-index rejection, serializable event histories, tamper evidence, and old-or-new crash recovery. It also exposes unavoidable limits: finite counters exhaust, bounded rollback forgets, finite tombstones cannot remember infinitely many deletions, and a genuine two-row merge is not one event.

The most important design choice is therefore not vector dimension but placement of authority. An alias map that bypasses row validation is another authority. An in-place neural refresh is another version. An uncharged deletion ledger defeats a total-system bound. A soft mixture is not a discrete fact version. Once those hidden states are made explicit, editable reasoning memory becomes a falsifiable systems object.

The result is not evidence that editable reasoning works. It is a bounded contract on which such evidence can later be demanded.

AI-Assisted Tooling Disclosure

The authors used AI-assisted programming and writing tools for drafting, editing, figure generation, and build verification. The authors reviewed the manuscript artifacts and retain responsibility for their content.

Author Contributions

Junyi Zou and Avrova Donz contributed equally to this work. Both authors share responsibility for the research direction and manuscript. No corresponding-author designation is made.

References

- [1] Carsten Bormann and Paul Hoffman. Concise binary object representation (CBOR). Technical Report RFC 8949, RFC Editor, 2020. <https://www.rfc-editor.org/rfc/rfc8949.html>.
- [2] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001. https://doi.org/10.1007/3-540-44503-X_20.

- [3] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation*, pages 205–218, 2006. <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/>.
- [4] Joseph A. Goguen and José Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, pages 11–20, 1982. <https://doi.org/10.1109/SP.1982.10014>.
- [5] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural turing machines, 2014. <https://arxiv.org/abs/1410.5401>.
- [6] Alex Graves, Greg Wayne, Malcolm Reynolds, et al. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. <https://doi.org/10.1038/nature20101>.
- [7] Jim Gray. The transaction concept: Virtues and limitations (invited paper). In *Proceedings of the 7th International Conference on Very Large Data Bases*, pages 144–154. IEEE Computer Society, 1981. <https://www.vldb.org/conf/1981/P144.PDF>.
- [8] Maurice Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. <https://doi.org/10.1145/78969.78972>.
- [9] C. Mohan, Don Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. <https://doi.org/10.1145/128765.128770>.
- [10] National Institute of Standards and Technology. The keyed-hash message authentication code (HMAC). Technical Report FIPS PUB 198-1, U.S. Department of Commerce, 2008. <https://csrc.nist.gov/pubs/fips/198-1/final>.
- [11] National Institute of Standards and Technology. Secure hash standard (SHS). Technical Report FIPS PUB 180-4, U.S. Department of Commerce, 2015. <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>.
- [12] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks, 2014. <https://arxiv.org/abs/1410.3916>.
- [13] Junyi Zou and Avrova Donz. MMLA: How memory lets the past shape the future, 2026. Public V3, <https://arxiv.org/abs/2606.28876v3>.

A Additional Proofs and Formal Boundaries

A.1 Canonical serialization and bit equality

PROVED UNDER STATED ASSUMPTIONS

Lemma A.1 (Unique valid serialization). *Assume every variable-length region has an explicit length, deterministic normalization, fixed padding, fixed ordering, and zero reserved bytes. Then two valid parsed rows with the same typed field values have identical serialized bytes.*

Proof. Proceed through the schema regions in their fixed order. Fixed-width fields have a unique byte encoding by the schema’s endianness and numeric rules. A variable-width field has one normalized content sequence, one encoded length, the exact content bytes, and uniquely determined zero padding to the fixed region boundary. Arrays have fixed capacity and canonical entry order; unused entries are the unique empty encoding. Reserved bytes are zero. Each region is therefore identical, hence their concatenation is identical. $\square$

This lemma fails if map order, Unicode normalization, NaN payloads, or padding are unconstrained. A semantic equality relation over decoded objects is not enough for byte-addressed checksums.

A.2 Lifecycle reduction and exact NULL

PROVED UNDER STATED ASSUMPTIONS

Proposition A.2 (Row-local lifecycle reduction). *Every authorized lifecycle operation whose complete postcondition changes at most one row and yields a valid schema row is representable as Eq. 30. Every rejected operation is representable as Eq. 31.*

Proof. For a valid row-local postcondition, let a be its unique changed row and let $\hat{\tau}_a$ be the complete postcondition serialization. Equation 30 produces exactly that table. A rejected operation has no legal postcondition and the contract assigns it NULL, which is state identity. $\square$

The premise “changes at most one row” excludes a genuine two-row merge, as Proposition 7.2 shows.

PROVED UNDER STATED ASSUMPTIONS

Proposition A.3 (Zero write is not NULL). *If a legal candidate encodes a typed zero value and advances slot sequence or receipt, its commit is not NULL even when every byte in its active payload region is zero.*

Proof. NULL leaves every authoritative byte unchanged. The legal zero candidate contains a new slot sequence and receipt by premise, so its serialized row differs from its predecessor. Its replacement therefore changes authoritative state. $\square$

A.3 Tombstone suppression

PROVED UNDER STATED ASSUMPTIONS

Theorem A.4 (Stale-value suppression during retained tombstone scope). *Assume the current row for object o is a valid retained TOMBSTONE row; reads require current LIVE status; every index hit is revalidated; and becoming LIVE requires an authorized version-advancing resurrection or rollback. Then no stale pre-deletion row or index entry can supply o ’s active content before such an authorized commit.*

Proof. The current row fails the LIVE predicate and therefore supplies no active content. A stale index entry for the pre-deletion row carries an earlier slot sequence or receipt digest, so Theorem 9.2 rejects it. Any attempt to replace the tombstone without resurrection/rollback authority fails validation and resolves to NULL, leaving the tombstone current. Thus only an authorized version-advancing commit can make o readable. $\square$

The scope ends when the tombstone is purged or when external ledgers no longer remember the deleted identity. The theorem does not promise perpetual suppression over an unbounded universe.

A.4 Tenant-local capacity

PROVED UNDER STATED ASSUMPTIONS

Corollary A.5 (Tenant resident capacity). *For tenant t with $S_t = |P^{-1}(t)|$, its authoritative row projection has at most $2^{8S_t B_R}$ bit states and carries at most $8S_t B_R$ bits of mutual information about any history.*

Proof. Apply Theorem 3.1 to the concatenation of the tenant’s S_t fixed-width rows. $\square$

This is both a storage bound and a denial-of-capacity boundary: a tenant cannot borrow another tenant’s rows under static partitioning.

A.5 Index rebuild determinism

PROVED UNDER STATED ASSUMPTIONS

Proposition A.6 (Deterministic base-index rebuild). *Under deterministic parsing, row validation, key hashing, quantization, and fixed slot-major output order, rebuilding I_C and I_N from the same authoritative table yields byte-identical indexes.*

Proof. Each output region is assigned to one fixed pair (a, q) for slot a and primary/alias ordinal q , or to one fixed slot a for the neural index. The validated row at that slot is fixed. Every projected field and digest is deterministic, and invalid/nonreadable projections have a unique empty encoding. Fixed output order eliminates scheduling dependence. Thus every output byte is a deterministic function of M . $\square$

A.6 Segment rank counterexample in matrix form

Let $S \geq 2$ and $d_v \geq 2$. Choose two distinct rows $a \neq b$ and payload changes $u, w \in \mathbb{R}^{d_v}$ that are linearly independent. Event 1 changes only row a by u ; event 2 changes only row b by w . Then

$$V(M_{n,2}) - V(M_{n,0}) = e_a u^\top + e_b w^\top, \quad (65)$$

where e_a, e_b are distinct standard basis vectors. The two nonzero rows are linearly independent, so the matrix rank is two. This construction satisfies every per-event rank-one bound and disproves an unconditional whole-segment rank-one claim.

A.7 Read-time fail-closed property

PROVED UNDER STATED ASSUMPTIONS

Proposition A.7 (No view selection after mismatch). *If Eq. 28 is the only active-content read rule, a row with checksum, authentication, status, ACL, time, or dual-view failure supplies neither canonical nor neural active content.*

Proof. The rule is a conjunction. Failure of any conjunct selects the otherwise branch $\perp$. No rule returns one view conditional on failure of the other. $\square$

The proposition is only as strong as the carrier inventory. An undocumented vector cache that bypasses Eq. 28 is an additional authority and invalidates the premise.

A.8 Commit and audit binding

PROVED UNDER STATED ASSUMPTIONS

Proposition A.8 (No accepted unaudited commit under a required-ledger profile). *Assume policy marks an event as requiring audit, the candidate receipt binds the intended audit sequence/root, and publication verifies that the prepared audit record is durably reachable from that root before changing the table selector. Then an accepted recovered postcommit root has the corresponding audit binding; failure to prepare the audit record yields the old root.*

Proof. If audit preparation fails, the publication precondition is false and the selector remains on the old root. If the selector reaches the new root, the protocol’s ordering premise guarantees the bound audit object was prepared and recovery validates its reachability. Therefore every accepted postcommit root satisfies the required binding. $\square$

This result binds an audit record, not an unbounded human-readable log. A bounded checkpoint/rotation policy determines how long detail remains available.

A.9 Assumption sensitivity

Dropped premise	Formal result lost	Minimal witness
Fixed field widths	capacity theorem	append one alias per event
Trusted field derivation	isolation/version/protection	proposal self-assigns tenant or version
Deterministic encoding	byte equality/rebuild	map order or NaN payload differs
Snapshot-consistent global checks	identity uniqueness	concurrent duplicate inserts
Serializable publication	serial history and uniqueness	write skew across target rows
Atomic root publication	no-torn/crash theorem	visible partial page or mixed root
Current-row index validation	stale suppression	old alias hit returns old row
Cryptographic assumptions	tamper-evidence theorem	collision or tag forgery
Static partition/logical observation	tenant noninterference	row migration or timing channel
Required-ledger fail-closed rule	audit binding	row publishes after audit failure

Table 7: Premises are necessary to the indicated claims; later evidence must test them independently.

B AMR-1 Profile Ledger

This appendix reconciles every byte used by the illustrative profile. It is a design artifact, not a deployed wire-format standard.

B.1 Trusted control: 256 bytes

Field	Bytes
Magic, schema, status, operation, flags	10
Physical slot identifier	4
Tenant identifier	16
Logical object identifier	16
Owner/principal identifier	16
Object type	2
Slot sequence	8
Object version	8
Valid-from, valid-until, retention-until	24
Protection class and deletion-reason code	4
Eight ACL entries: 16-byte principal + 2-byte rights	144
Rollback ring head and occupancy	2
Reserved zero bytes	2
Total	256

Table 8: AMR-1 trusted-control allocation.

The object type and header flags supply the canonical type/semantic flags referenced in Eq. 8; they remain system-owned.

B.2 Canonical capsule: 1,104 bytes

$$\underbrace{(2 + 64)}_{\text{key}} + \underbrace{(2 + 512)}_{\text{value}} + \underbrace{(1 + 3) + 8(1 + 64)}_{\text{alias count/reserved + entries}} = 1,104. \quad (66)$$

Lengths count canonical bytes, not code points. Padding to each maximum is zero. The primary key and value use 16-bit lengths; each alias uses an 8-bit length because its maximum is 64. Alias count is at most eight.

B.3 Neural capsule: 784 bytes

$$\underbrace{128 \times 2}_{\text{routing}} + \underbrace{256 \times 2}_{\text{payload}} + \underbrace{4}_{\text{uncertainty + abstention}} + \underbrace{1}_{\text{proposed intent}} + \underbrace{11}_{\text{reserved}} = 784. \quad (67)$$

The 768 routing/payload bytes participate in exact derived-mode comparison. Uncertainty and abstention are two signed or unsigned 16-bit schema values; intent is one enumerated byte.

B.4 Provenance: 192 bytes

Field	Bytes
Event identifier	16
Source-material or source-receipt digest	32
Proposer/model-build digest	32
Assembler build identifier	16
Encoder build identifier	16
Validator build identifier	16
Observation time	8
Source class	2
Committed operation copy	1
Deletion-authorization digest	32
External audit handle	16
Reserved zero bytes	5
Total	192

Table 9: AMR-1 bounded provenance allocation. A source document or provenance graph is external.

B.5 Rollback: two 2,048-byte snapshots

Each snapshot contains no nested rollback data:

Field	Bytes
Prior canonical capsule	1,104
Prior neural capsule	784
Same-object identifier	16
Prior slot sequence	8
Prior object version	8
Prior validity and retention times	24
Prior protection/status/snapshot flags	8
Prior provenance digest	32
Prior receipt digest	32
Reserved zero bytes	32
One snapshot	2,048
Two-snapshot ring	4,096

Table 10: AMR-1 rollback allocation. The snapshot is sufficient to propose restoration of semantic state; current policy and new provenance are assembled at rollback time.

An evicted different object is excluded. A rollback snapshot does not carry an ACL or recursively embedded history; current security controls govern restoration.

B.6 Consistency receipt: 160 bytes

Field	Bytes
Core checksum/digest	32
Predecessor receipt digest	32
Validation bitmap	8
Schema/algorithm/key-epoch/format identifiers	16
Event binding	16
Authentication tag	32
Reserved zero bytes	24
Total	160

Table 11: AMR-1 receipt allocation.

The receipt-covered core is therefore $6,592 - 160 = 6,432$ bytes.

B.7 Base-index entries

One 112-byte canonical entry contains:

$$16_{\text{tenant}} + 16_{\text{object}} + 32_{\text{key digest}} + 4_{\text{slot}} + 8_{\text{sequence}} + 32_{\text{receipt digest}} + 4_{\text{ordinal/flags}} = 112. \quad (68)$$

One 336-byte neural entry contains:

$$256_{\text{route}} + 16_{\text{tenant}} + 16_{\text{object}} + 4_{\text{slot}} + 8_{\text{sequence}} + 32_{\text{receipt digest}} + 4_{\text{flags}} = 336. \quad (69)$$

The fixed slot-major arrays contain entries for empty/tombstoned/quarantined rows in the unique disabled normal form. Their size is therefore independent of occupancy.

B.8 Auxiliary and external carrier parameters

AMR-1’s reported 1.9375 MiB total includes authoritative rows and both base indexes. It sets persistent non-authoritative fast state and diagnostic-copy quarantine to zero. A conforming extension publishes the following additional parameters:

$$\begin{aligned}
 B_Q &= Q_{\max} B_q && \text{diagnostic copies,} \\
 B_F &= S_F d_F p_F + B_{F,\text{meta}} && \text{soft fast state,} \\
 B_A(t) &\leq A_{\max} \text{ or explicitly unbounded} && \text{archive,} \\
 B_L(t) &\leq L_{\max} B_\ell + B_{L,\text{index}} \text{ or explicitly unbounded} && \text{audit ledger.}
 \end{aligned} \quad (70)$$

If B_Q or B_F is nonzero, its authority and reset/lifetime rule must be audited. If $A_{\max}$ or $L_{\max}$ is finite, reaching it invokes Table 2; if absent, total-system boundedness is not claimed.

The safe-path ephemeral working set has at least the proposal, candidate, and encoder scratch regions,

$$B_{\text{work,min}} = 1,888 + 6,592 + 768 = 9,248 \text{ bytes}, \quad (71)$$

plus implementation-specific old-row, parser, hash/MAC, policy, index, and I/O buffers. A deployment must measure its peak physical working set rather than treating the minimum as exact RAM use.

B.9 Arithmetic reconciliation

$$256 + 1,104 + 784 + 192 + 4,096 + 160 = 6,592. \quad (72)$$

$$6,592 \times 256 = 1,687,552, \quad 256(9 \times 112 + 336) = 344,064. \quad (73)$$

$$1,687,552 + 344,064 = 2,031,616 = 1.9375 \times 2^{20}. \quad (74)$$

Every headline AMR-1 byte count follows from these fixed allocations.

C Theorem-to-Implementation Obligations

Theorems are reusable only when the implementation supplies the corresponding premise evidence. Table 12 is a minimum receipt set.

Table 12: Formal result to implementation and falsification obligations.

Claim	Required premise bundle	Minimum implementation receipt/test
Fixed resident capacity (Theorem 3.1)	exact schema; complete carrier inventory; fixed S, B_R	schema-generated offset table; binary-size checks; allocator and external-carrier inventory; exhaustion tests
Unique serialization (Lemma A.1)	deterministic normalization, lengths, padding, order, numeric rules	cross-language golden corpus and boundary/fuzz vectors with byte-identical results
Event/segment locality (Theorem 6.2)	all authority in M ; one target or NULL per event	whole-process write tracing; state hash before/after every event; hidden-cache/alias-map inventory
Trusted-field nondelegation (Proposition 4.1)	proposer isolated from trusted functions and keys	data-flow/code audit; privilege separation; adversarial field-injection suite; key-access audit
No torn dual view (Theorem 5.2)	one atomic complete-row publication	crash/concurrency injection at every row-write boundary; no mixed serialization accepted
Invariant preservation (Theorem 8.11)	valid genesis; trusted deterministic assembly; snapshot-consistent serializable validation	model check on reduced schema; property-based transition tests; global-uniqueness concurrency tests
Version monotonicity (Corollary 8.12)	exact increment and overflow rejection	long trace; stale replay; maximum counter; restart and recovery tests
Untouched preservation (Corollary 8.13)	complete target isolation	byte hashes of all non-target rows across every event and crash point
Monotone rollback (Proposition 7.1)	same-object bounded snapshot; current security; version increment	rollback/resurrection adversary; obsolete ACL/protection fixtures; ring-overflow tests
Tombstone suppression (Theorem A.4)	retained current tombstone; revalidated indexes; authorized resurrection only	stale row/index replay, aliases, backups in scope, privilege-negative tests
Stale-index safety (Theorem 9.2)	every hit bound to and checked against current row	fault injection dropping/reordering index updates; slot reuse; cross-tenant forged entries
Tenant noninterference (Theorem 9.1)	static partitions and logical observation model	cross-tenant mutation/read adversary; separate side-channel scope statement and tests
Serializable history (Theorem 9.4)	tenant-total order or equivalent CAS; global constraint serialization	concurrent history checker; write-skew fixtures; linearization receipt; stale snapshot rejection
Old-or-new recovery (Theorem 9.6)	correct storage protocol, atomic selector, durable ordering, validating recovery	power/process/device crash matrix; torn selectors/pages; flush/fence audit; replica failover if claimed

Claim	Required premise bundle	Minimum implementation receipt/test
Receipt tamper evidence (Theorem 8.16)	collision resistance, unforgeability, protected keys	algorithm/key inventory; bit-flip/replay/substitution suite; rotation/revocation and compromise response
Index rebuild determinism (Proposition A.6)	deterministic projection and fixed order	repeated parallel/serial/cross-language rebuilds with identical hashes
Audit binding (Proposition A.8)	required-ledger reservation and atomic root binding	audit-full, write-failure, rotation, recovery, and dangling-record tests
Semantic usefulness	<i>not implied by any theorem</i>	preregistered same-checkpoint reconstruction/query/calibration/preservation qualification
Reasoning or cost advantage	<i>not implied by any theorem</i>	repeated-write, security, and total-system compute/storage/latency/energy matched comparison

C.1 Implementation conformance statement

DESIGN PRESCRIPTION A system claiming AMR conformance should publish:

1. schema identifier and generated field-offset/size table;
2. complete mutable-carrier inventory and authority classification;
3. trusted computing boundary, privilege/key map, and build digests;
4. serializer/parser/validator golden vectors;
5. event trace with snapshot, candidate, decision, target, predecessor, new receipt, and NULL reason;
6. current-row read and index-revalidation trace;
7. tenant/ACL/protection/deletion/rollback policy fixtures;
8. storage publication ordering and recovery specification;
9. archive, audit, index, quarantine, replication, and backup capacity/retention policies;
10. exact logical and measured physical cost ledger; and
11. explicit list of theorems claimed, with every premise receipt linked.

Missing evidence weakens only the affected claim. For example, a correct serializer may support fixed-bit parsing while crash recovery remains unvalidated. A successful semantic reader may support interface usefulness while deletion authority remains unvalidated. Status must not be inherited by the whole system.

C.2 Stop-rule ordering

The intended order is:

$$\begin{aligned}
 &\text{schema/transition mechanics} \rightarrow \text{serialization and model checking} \rightarrow \text{security/concurrency} \\
 &\rightarrow \text{crash recovery and exact cost} \rightarrow \text{semantic interface} \rightarrow \text{learned admission} \\
 &\rightarrow \text{repeated writes} \rightarrow \text{total-system quality--cost comparison.}
 \end{aligned} \tag{75}$$

Failure at a rung stops claims that depend on it. It does not erase independent lower-rung evidence, and it does not authorize a post-hoc threshold, seed, checkpoint, or threat-model change.

The frozen MMLA result currently stops at the semantic-interface rung for the tested interfaces. This manuscript supplies a new formal substrate candidate, not evidence that the rung has been passed.

D Symbol and Status Ledger

Table 13: Principal symbols.

Symbol	Meaning
σ	immutable row schema and deterministic serialization profile
$S, \mathcal{S}$	resident slot count and slot set $\{1, \dots, S\}$
M, r_a	complete authoritative table and row in slot a
B_R	complete serialized row bytes; AMR-1 uses 6,592

Symbol	Meaning
$h, c, n, p, \lambda, \rho$	trusted control, canonical, neural, provenance, rollback, and receipt row blocks
$P(a)$	trusted static tenant assigned to slot a
v_s, v_o	physical slot sequence and same-object version
$z(r)$	row status: EMPTY, LIVE, TOMBSTONE, or QUARANTINED
$\tilde{p}$	untrusted neural proposal
χ	authenticated request/event context and capabilities
$D_{\sigma, \omega}$	versioned canonical–neural conformance relation
NULL	exact no-authoritative-change action
E_n, K_n	ordered event list for segment n and its length
$M_{n,j}$	authoritative state after event j of segment n
C_n, D_n	accepted commit count and distinct changed-slot count in segment n
B_I, B_Q, B_F	fixed index, diagnostic quarantine, and non-authoritative fast-state capacity
$B_A(t), B_L(t)$	separately charged archive and audit-ledger storage at time t
$\mathcal{W}$	heterogeneous logical work vector for a candidate/commit
B_P, B_C, B_D	proposal, canonical-scan, and compared neural route/payload bytes
N_H	hash compression-block count
Obs_t	logical authorized observation projected to tenant t
Inv_σ	conjunction of the sixteen state invariants

D.1 Status-to-evidence rule

Displayed status	Admissible support in this paper	Required before promotion
DEFINITION	internal typed construction	independent terminology and interface audit
PROVED UNDER STATED ASSUMPTIONS	deductive proof in source	independent proof audit; premise evidence for implementation use
DESIGN PRESCRIPTION	engineering necessity/counterexample	implemented mechanism and adversarial test
PLANNED TEST	protocol text only	immutable preregistration and completed evidence
UNVALIDATED	explicit absence of qualifying evidence	matched fresh evaluation under frozen gates
CONJECTURED	stated hypothesis only	theory proof or accepted experiment, depending on claim

Table 14: No status automatically promotes another.

D.2 Artifact boundary

The mathematical statements in this paper are candidate theory. Project-result statements are limited to the public V3 and technical-report evidence boundary. Source organization and arithmetic are not treated as scientific evidence without an explicit derivation or admitted measurement.